Android新增Camera驱动开发指南

文件标识：RK-KF-YF-E50

发布版本：V2.1.0

日期：2026-03-20

文件密级：□绝密 □秘密 □内部资料 ■公开

免责声明

本文档按“现状”提供，瑞芯微电子股份有限公司（“本公司”，下同）不对本文档的任何陈述、信息和内容的准确性、可靠性、完整性、适销性、特定目的性和非侵权性提供任何明示或暗示的声明或保证。本文档仅作为使用指导的参考。

由于产品版本升级或其他原因，本文档将可能在未经任何通知的情况下，不定期进行更新或修改。

商标声明

“Rockchip”、“瑞芯微”、“瑞芯”均为本公司的注册商标，归本公司所有。

本文档可能提及的其他所有注册商标或商标，由其各自拥有者所有。

版权所有 © 2026瑞芯微电子股份有限公司

超越合理使用范畴，非经本公司书面许可，任何单位和个人不得擅自摘抄、复制本文档内容的部分或全部，并不得以任何形式传播。

瑞芯微电子股份有限公司

Rockchip Electronics Co., Ltd.

地址： 福建省福州市铜盘路软件园A区18号

网址： www.rock-chips.com

客户服务电话： +86-4007-700-590

客户服务传真： +86-591-83951833

客户服务邮箱： fae@rock-chips.com

前言

本文档是基于RK ANDROID平台移植、开发摄像头驱动的帮助文档。

概述

本文档从Camera类型介绍、Camera调试评估、Rockchip Camera数据流框图、Camera调试准备工作，及Camera调试步骤这个几个方面，介绍如何在Rockchip的芯片平台新增移植调试一个摄像头驱动。

产品版本

芯片名称	内核版本	**Android**版本
RK3326	Linux 4.19	Android11.0
RK3368	Linux 4.19	Android11.0
RK3288	Linux 4.19	Android11.0
RK3399	Linux 4.19	Android11.0
RK3566/RK3568	Linux 4.19	Android11.0
RK3588	Linux 5.10/Linux 6.1	Android12.0/13/14
RK3562	Linux 5.10/Linux 6.1	Android13/14
RK3576	Linux 6.1	Android14
RK3572	Linux 6.12	Android16

读者对象

本文档（本指南）主要适用于以下工程师：

技术支持工程师

软件开发工程师

修订记录

版本号	作者	修改日期	修改说明
V1.0.0	王潘祯撰	2025-02-10	初始版本
V2.0.0	范建威	2025-04-18	补充芯片规格、移植步骤、常用调试方法与常见问题排查； 修复相关错误描述。
V2.0.1	范建威	2025-05-23	增加章节AHD多路摄像头描述
V2.1.0	范建威	2026-03-20	增加RK3572描述

目录

Android新增Camera驱动开发指南

1. 常用Camera类型介绍
    - 1.1 USB camera
    - 1.2 DVP camera
    - 1.3 MIPI camera
    - 1.4 本文所指Camera驱动移植范围
2. Camera调试评估
    - 2.1 SOC ISP规格
    - 2.2 SOC接口规格
    - 2.3 DCPHY 注意事项
3. Camera数据流框图
    - 3.1 ISP1.0数据流框图
    - 3.2 ISP2.x及ISP3.x数据流框图
4. Camera调试准备工作
5. Camera移植步骤
    - 5.1 驱动移植
        - 5.1.1 标准I2C子设备驱动实现
        - 5.1.2 v4l2子设备实现
        - 5.1.3 初始化配置序列
        - 5.1.4 上电时序
        - 5.1.5 下电时序
        - 5.1.6 曝光增益等控制实现
        - 5.1.7 MIPI时钟与像素时钟配置
        - 5.1.8 HDR模式调试
        - 5.1.9 PDAF添加
        - 5.1.10 OTP添加
        - 5.1.11 虚拟通道配置
        - 5.1.12 MIPI与并口配置
        - 5.1.13 VCM驱动移植
        - 5.1.14 参考文档
    - 5.2 dts链路配置说明
        - 5.2.1 设备配置
            - 5.2.1.1 MIPI sensor
            - 5.2.1.2 DVP sensor
            - 5.2.1.3 VCM设备
            - 5.2.1.4 OTP设备
            - 5.2.1.5 HDR配置
            - 5.2.1.6 PDAF配置
        - 5.2.2 链路配置
            - 5.2.2.1 不同平台的数据链路
                - 5.2.2.1.1 ISP平台
                - 5.2.2.1.2 ISP2.x平台
                - 5.2.2.1.3 ISP3.x
                - 5.2.2.1.4 DPHY与DCPHY
            - 5.2.2.2 RK3588
            - 5.2.2.3 RK3588S2说明
            - 5.2.2.4 RK3576
            - 5.2.2.5 RK3572
            - 5.2.2.6 RK3562
            - 5.2.2.7 RK3568/RK3566
            - 5.2.2.8 RK3399/RK3288/RK3326
            - 5.2.2.9 回读模式与直通模式
    - 5.3 驱动功能验证
        - 5.3.1 media拓扑确认
        - 5.3.2 v4l2输出数据流

5.3.3 v4l2抓图验证
5.4 camera3_profiles.xml注册文件配置
5.4.1 设备注册名称与ID
5.4.2 分辨率配置
5.4.3 前后摄配置
5.4.4 旋转配置
5.4.5 SOC模式与RAW模式
5.4.6 HDR设置
5.4.7 多摄配置
5.5 初始效果文件生成
5.5.1 生成初始效果文件
5.5.2 分辨率、LSC修改
5.5.3 sensor info参数
5.6 GKI版本说明
5.6.1 驱动ko加载顺序
5.7 AHD/SERDES多路摄像头调试说明
5.7.1 驱动移植注意点
5.7.2 热拔插机制
5.7.2.1 AHD驱动实现热拔插检测
5.7.2.2 VICAP复位机制
5.7.2.2.1 kernel-4.19
5.7.2.2.2 kernel-5.10/kernel-6.1
5.7.3 dts配置注意点
5.7.3.1 热拔插配置
5.7.3.2 适配cameraHAL
5.7.4 external cameraHAL配置
6. 常用调试命令与调试工具
6.1 打开debug log开关
6.1.1 打开ISP驱动debug
6.1.2 打开VICAP驱动debug
6.1.3 打开cameraHAL debug
6.1.4 打开rkaiq debug
6.2 查看版本号
6.3 常用V4L2命令
6.3.1 查看所有节点
6.3.2 抓图
6.3.2.1 VICAP输出raw
6.3.2.2 ISP maipath输出非紧凑raw
6.3.2.3 VICAP输出YUV
6.3.2.4 ISP输出YUV
6.3.3 输出数据流
6.3.4 查看链路
6.3.5 查看sensor所有控制项
6.3.6 配置数据链路
6.3.7 配置ISP输出分辨率格式
6.3.8 手动设置曝光增益
6.3.9 手动设置马达
6.4 查看ISP/CIF状态命令
6.5 查看注册camera信息
6.6 cameraHAL抓图
6.7 查看camera预览帧率
6.8 io命令使用
6.9 raw/yuv图预览查看工具
6.10 rkaiq_demo抓图
6.11 3A server抓图
7. 常见问题排查方法
7.1 I2C通讯异常排查
7.2 APK打不开预览排查

7.3 MIPI异常报错排查
7.4 注册不上摄像头排查
7.5 多摄配置打开异常
7.6 常见调试问题参考文档

1. 常用Camera类型介绍

按照接口类型分类有USB camera、MIPI camera、DVP camera。

1.1 USB camera

USB Camera常见的数据格式有：YUYV/MJPEG/H.264

YUYV：传输的是YUV数据，数据量很大，分辨率和帧率较低，一般由后端对数据进行再处理，用于一些特殊用途（虹膜、红外人脸识别等）。

MJPEG：一般用于分辨率相对较低的USB Camera（720P,VGA）,传输数据较大。

H.264：一般用于分辨率较高的USB Camera（1080p,720p），传输数据较小。

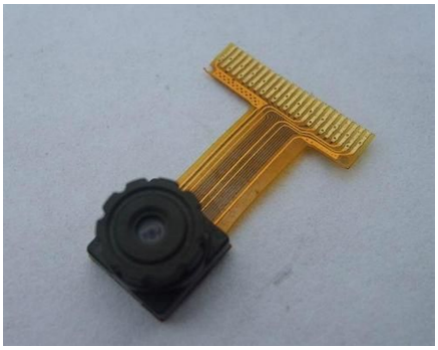
示例USB camera如图：

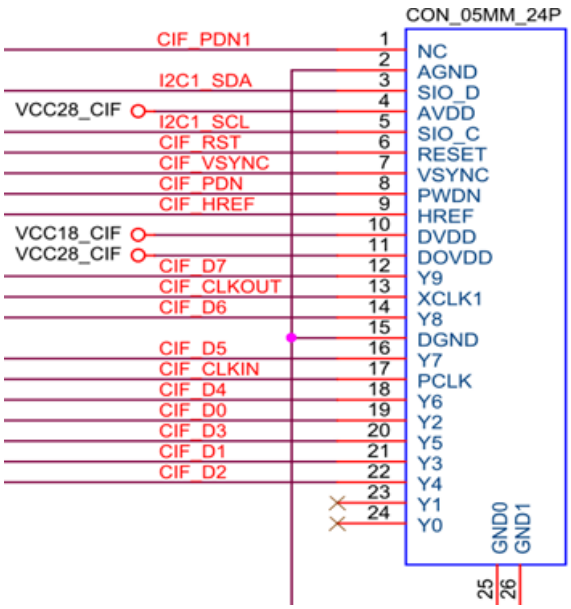


1.2 DVP camera

DVP Camera或称为并口Camera，接口如下图所示，一般支持BT601/BT656/BT1120数据的传输。DVP是并口传输，速度相对较慢，传输的带宽较低，一般用于500万像素以下的camera。需要使用PCLK时钟、VSYNC场同步、HSYNC行同步、D[0：11]并口数据，可以是8/10/12bit/16bit数据位大小。

示例DVP camera及接口如图：






```
5%  R0402   CIF_D0/EBC_SDDO0/VOPBT656_D0_M1
5%  R0402   CIF_D1/EBC_SDDO1/VOPBT656_D1_M1
5%  R0402   CIF_D2/EBC_SDDO2/VOPBT656_D2_M1
5%  R0402   CIF_D3/EBC_SDDO3//VOPBT656_D3_M1
5%  R0402   CIF_D4/EBC_SDDO4/VOPBT656_D4_M1
5%  R0402   CIF_D5/EBC_SDDO5/VOPBT656_D5_M1
5%  R0402   CIF_D6/EBC_SDDO6/VOPBT656_D6_M1
5%  R0402   CIF_D7/EBC_SDDO7/VOPBT656_D7_M1

5%  R0402   CIF_D8/EBC_SDDO8/GMAC1_TXD2_M1
5%  R0402   CIF_D9/EBC_SDDO9/GMAC1_TXD3_M1
5%  R0402   CIF_D10/EBC_SDDO10/GMAC1_TXCLK_M1
5%  R0402   CIF_D11/EBC_SDDO11/GMAC1_RXD2_M1
5%  R0402   CIF_D12/EBC_SDDO12/GMAC1_RXD3_M1
5%  R0402   CIF_D13/EBC_SDDO13/GMAC1_RXCLK_M1
5%  R0402   CIF_D14/EBC_SDDO14/GMAC1_TXD0_M1
5%  R0402   CIF_D15/EBC_SDDO15/GMAC1_TXD1_M1
```

1.3 MIPI camera

MIPI为低压差分信号，传输速度快，抗干扰能力强，一般支持800万及以上像素的camera。目前主流手机camera模组都使用MIPI传输，传输时使用4对差分数据信号和1对差分时钟信号，MIPI接口如下图所示。

示例MIPI Camera如图：



1.4 本文所指Camera驱动移植范围

本文所指新增的Camera，指的是MIPI或者DVP接口的Camera，可以是直接CMOS Sensor输出MIPI/DVP接口信号，连接到主控，也可以是从AHD转换芯片出来的MIPI/DVP 接口信号，连接到主控AP。

2. Camera调试评估

新增调试一个Camera前，需要先评估当前想应用的芯片平台是否支持该Camera。一般需要考量的是当前平台支持的格式、最大分辨率、帧率及最多支持几个camera等。

2.1 SOC ISP规格

如下表格大致列出目前几个常用平台支持的相关参数。

芯片名称	ISP支持最大规格	ISP个数	CIF个数	DVP接口	最大支持camera数
RK3326	8M：3264x2448@30FPS	1	1	1	2
RK3368	8M：3264x2448@30FPS	1	1	1	2
RK3288	13M：4416x3312@15FPS, 8M：3264x2448@30FPS	1	1	1	2
RK3399	13M：4416x3312@15FPS, 8M：3264x2448@30FPS	2	0	0	2
RK3568	8M：3264x2448@30FPS	1	1	1	3
RK3588	48M： 8064x6048@15 dual isp 32M：6528x4898@30 dual isp 16M：4672x3504@30 single isp	2	1	1	7
RK3562	13M：4224x3136@30FPS	1	1	0	4
RK3576	48M：8064x6048@10FPS 16M：4672x3504@30FPS	1	1	1	6
RK3572	12M：4096x3072@30FPS	1	1	1	5

一般支持的输入的数据格式：RAW8/10/12 YUV422 等，各个芯片平台有点差异，具体见芯片Datasheet。

若是YUV422格式输入的场景，RK3588/RK3562/RK3576/RK3572等可不需要ISP处理，最大支持可以达到4K60FPS。

2.2 SOC接口规格

主控的接收能力除了ISP规格，还受限于接口的规格，主要有MIPI接口和DVP接口，以下是常用主控平台MIPI-CSI规格：

芯片型号	**MIPI RX数量 FULL MODE/SPLIT MODE**	**PHY规格**
RK3568/6	**1路**：1x4lane **2路**： 2x2lane	1xDPHY-v1.2: 2.5G/lane x 4lane
RK3588s	**3路**：3x4lane **4路**：2x2lane + 2x4lane	DPHY: 1xDPHY-v1.2: 2.5G/lane x 4lane DCPHY: (1)DPHY: 2xDPHY-v2.0: 2.5Gbps/lane x 4lane (2)CPHY: 2xCPHY-v1.1: 2.5Gsps/trio(5.7Gbps) x 3trios
RK3588s2	**3路**：3x4lane **4路**：2x2lane + 2x4lane	DPHY: 2xDPHY-v1.2: 2.5G/lane x 4lane DCPHY: (1)DPHY: 1xDPHY-v2.0: 2.5Gbps/lane x 4lane (2)CPHY: 1xCPHY-v1.1: 2.5Gsps/trio(5.7Gbps) x 3trios
RK3588	**4路**：4x4lane **6路**：4x2lane + 2x4lane	DPHY: 2xDPHY-v1.2: 2.5G/lane x 4lane DCPHY: (1)DPHY: 2xDPHY-v2.0: 2.5G/lane x 4lane (2)CPHY: 2xCPHY-v1.1: 2.5Gsps/trio(5.7Gbps) x 3trios
RK3562	**2路**：2x4lane **4路**：4x2lane	2xDPHY-v1.2: 2.5G/lane x 4lane
RK3576	**3路**：3x4lane **5路**：4x2lane + 1x4lane	DPHY: 1xDPHY-v1.2: 2.5G/lane x 4lane DCPHY: (1)DPHY: 2xDPHY-v2.0: 2.5G/lane x 4lane (2)CPHY: 2xCPHY-v1.1: 2.5Gsps/trio(5.7Gbps) x 3trios
RK3572	**2路**：2x4lane **4路**：4x2lane	2xDPHY-v1.2: 2.5G/lane x 4lane

注：CPHY描述带宽吞吐率单位为sps，与bps的换算：1 Gsps = 2.28 Gbps。

HDMI To BT1120规格受限于PCLK大小，规格如下：

芯片型号	支持数量	PCLK
RK3588s	1	148.5M
RK3588	1	148.5M
RK3576	1	148.5M
RK3568/6	1	148.5M
RK3562	0	0
RK3572	1	148.5M

2.3 DCPHY 注意事项

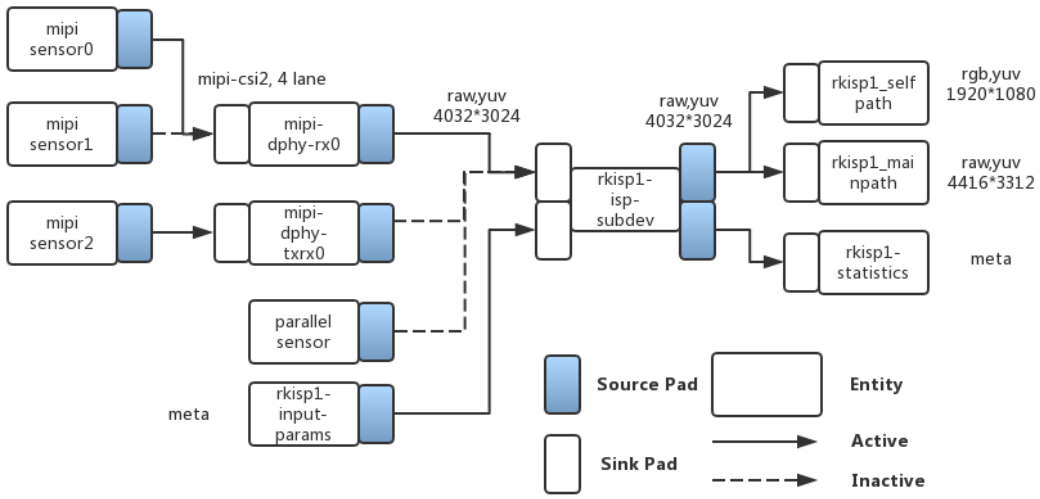
RK3588有2路DCPHY，RK3588S有2路DCPHY，RK3588S2有1路DCPHY，DCPHY对sensor输出的MIPI退出高速的LP11时序有严格的要求，要求必须按照lane3 > lane2 > lane1 > lane0的顺序退出，很多sensor难以满足这个要求，可能会导致概率性收不下数据，因此建议DCPHY仅在2lane的场景下使用。

RK3576 DCPHY则不要求上述事项。

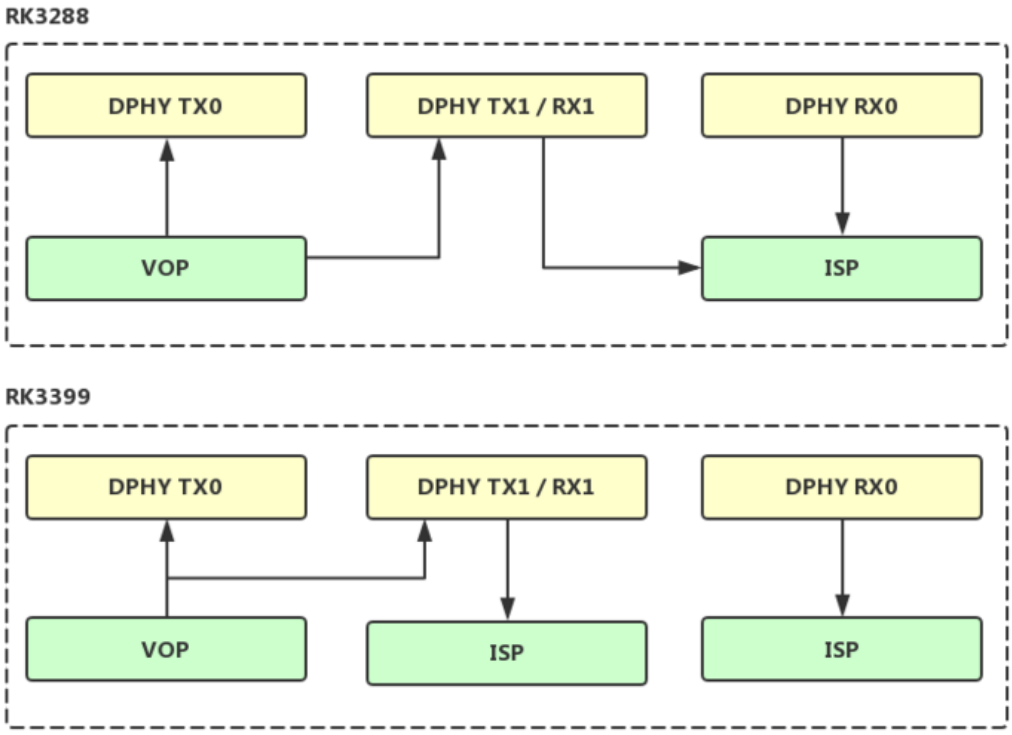
3. Camera数据流框图

3.1 ISP1.0数据流框图

RK3326/RK3368/RK3288/RK3399使用的是ISP1.0，其中RK3399的数据流示例框图如下：

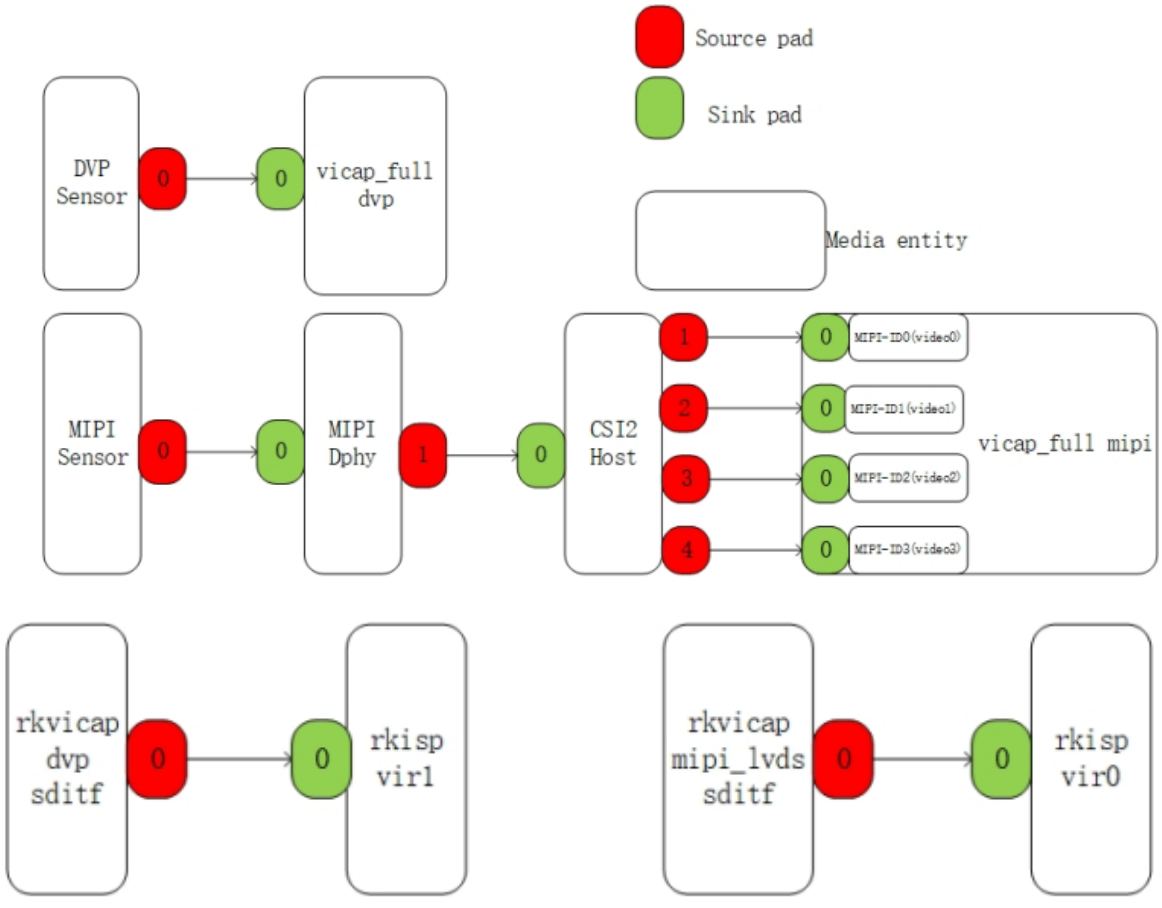


RK3288/RK3399硬件通路示例框图：

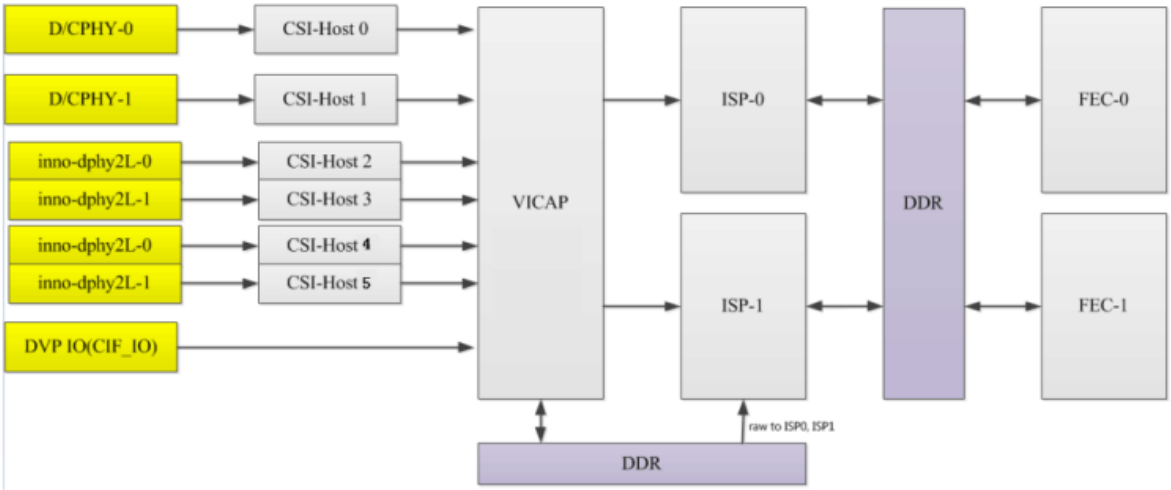


3.2 ISP2.x及ISP3.x数据流框图

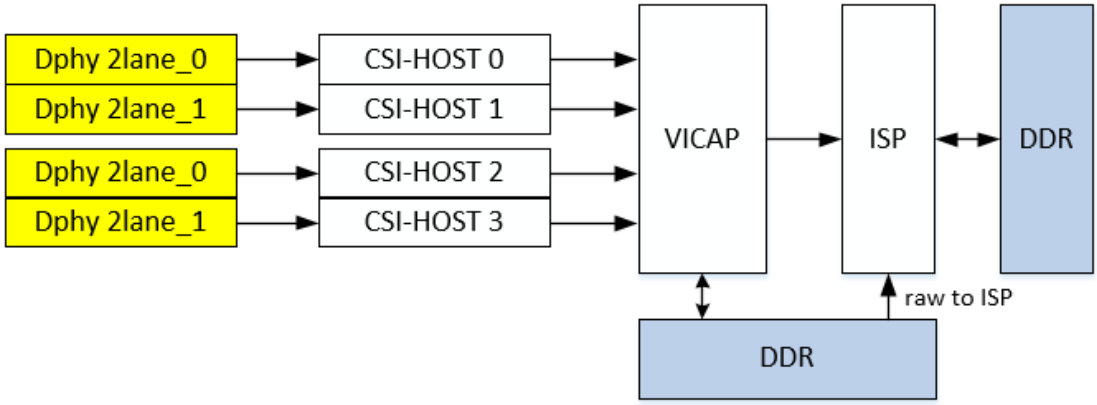
RK356x(ISP2X), RK3588 (ISP3x) ,RK3576 (ISP3x)的数据流程框图如下：



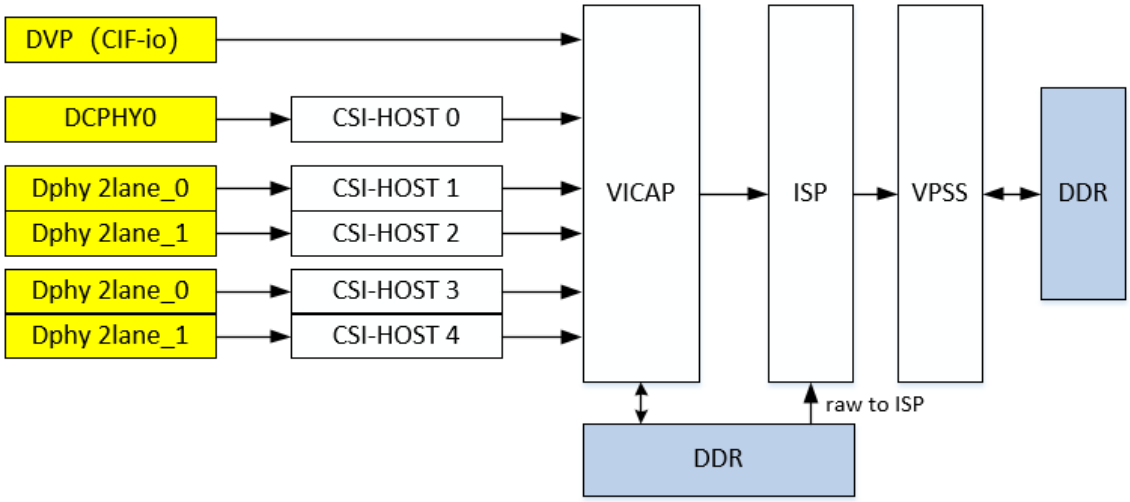
其中RK3588硬件通路示例框图如图：



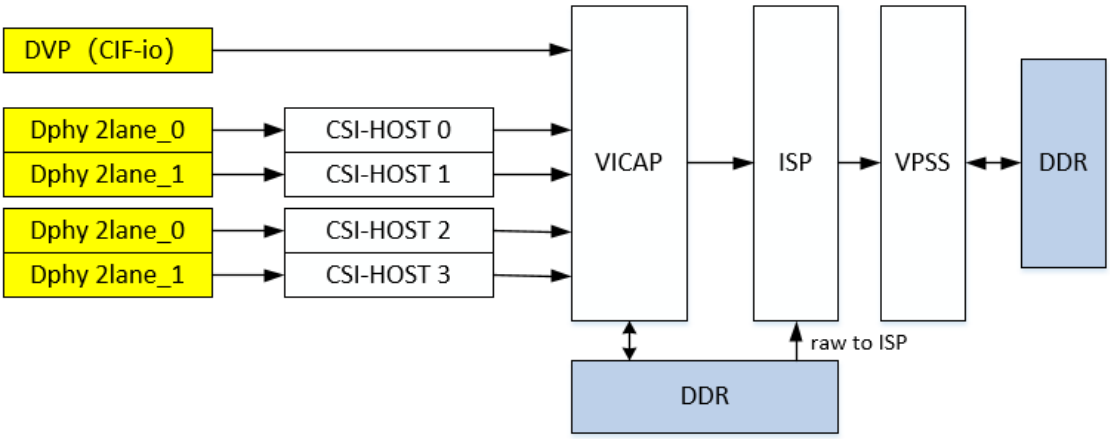
RK3562通路如下所示：



RK3576硬件通路如下所示：



RK3572硬件通路如下所示：


DVP （CIF-io）
Dphy 2lane_0
Dphy 2lane_1
Dphy 2lane_0
Dphy 2lane_1
CSI-HOST 0
CSI-HOST 1
CSI-HOST 2
CSI-HOST 3
VICAP
ISP
VPSS
DDR
raw to ISP
DDR

4. Camera调试准备工作

这节主要介绍新增camera调试所需准备的一些资料。

硬件原理图：

- 板子原理图及PCB位图（提供PDF文件）

Camera Module：

- 模组规格书；
- 镜头规格书；
- Camera Sensor Datasheet；
- Camera Sensor Application note；
- Camera Sensor 寄存器配置序列
- 如有其它平台调试过该sensor，获取其他平台驱动以供参考；

如果支持VCM，需有以下资料

- 马达驱动IC资料
- 马达规格书、
- 马达振荡曲线数据

如果支持OTP，需有以下资料

- OTP烧录规范

5. Camera移植步骤

5.1 驱动移植

Rockchip Android9.0以后的平台，camera都是基于V4L2框架实现。camera sensor驱动遵循V4L2框架和media framework框架。主控通过i2c与sensor进行交互，因此sensor对于主控来说是i2c子设备。所以sensor的驱动添加主要包括有：标准I2C子设备驱动部分实现、V4L2子设备实现、其他私有的流程实现。以下主要介绍一下具体实现过程。

5.1.1 标准I2C子设备驱动实现

1. 根据struct i2c_driver说明实现以下成员

```
a. struct driver.name
b. struct driver.pm
c. struct driver .of_match_table
d. probe函数
e. remove函数
```

具体可参考media/i2c下的sensor驱动实现：

```
#if IS_ENABLED(CONFIG_OF)
static const struct of_device_id imx577_of_match[] = {
    { .compatible = "sony,imx577" },
    {},
};
MODULE_DEVICE_TABLE(of, imx577_of_match);
#endif

static const struct i2c_device_id imx577_match_id[] = {
    { "sony,imx577", 0 },
    {},
};

static struct i2c_driver imx577_i2c_driver = {
    .driver = {
        .name = IMX577_NAME,
        .pm = &imx577_pm_ops,
        .of_match_table = of_match_ptr(imx577_of_match),
    },
    .probe      = &imx577_probe,
    .remove     = &imx577_remove,
    .id_table   = imx577_match_id,
};

static int __init sensor_mod_init(void)
{
    return i2c_add_driver(&imx577_i2c_driver);
}

static void __exit sensor_mod_exit(void)
{
    i2c_del_driver(&imx577_i2c_driver);
}
```

2. probe函数实现关键点

probe函数是完成驱动初始化注册的过程，实现的细节主要包括：获取DTS设备树定义的资源、v4l2子设备以及media实体的初始化。

probe函数中获取DTS配置的设备资源，主要包括如下资源。

成员名称	描述
CIS设备工作参 考时钟	采用外部独立晶振方案无需获取,RK参考设计一般采用AP输出时钟，该方案需 要获取，一般名称为xvclk
CIS设备控制GPIO	例如：Resst引脚, Powerdown引脚
CIS设备控制电源	根据实际硬件设计,获取匹配的软件电源控制资源,例如gpio,regulator
RK私有资源定义	命名方式按照rockchip,camera-module-xxx, 该部分资源会由驱动上传给用户态的camera_engine来决定IQ效果参数的匹配，安卓cameraHAL的camera ID的注册。

sensor驱动在probe函数中实现v4l2设备以及media实体的初始化

v4l2子设备：v4l2_i2c_subdev_init，RK CIS驱动要求subdev拥有自己的设备节点供用户态rk_aiq与安卓cameraHAL的访问，通过该设备节点实现曝光控制；
media实体：media_entity_init ，对接media framework，用于数据链路的控制。

5.1.2 v4l2子设备实现

v4l2子设备实现主要参考struct v4l2_subdev_ops说明，主要实现以下3个成员 ：

```
struct v4l2_subdev_core_ops
struct v4l2_subdev_video_ops
struct v4l2_subdev_pad_ops
```

1. v4l2_subdev_core_ops成员回调函数实现

主要实现以下回调：

成员名称	描述
.s_power	s_power实现sensor的上下电操作，对于一些寄存器数组较长的sensor，可以归纳出公共部分的寄存器，在上电后执行，s_stream只配置不同分辨率配置必须的寄存器，加快切换速度。
.ioctl	ioctl主要实现的RK私有控制命令
.compat_ioctl32	ioctl主要实现的RK私有控制命令

其中ioctl一般涉及到如下控制：

成员名称	描述
RKMODULE_GET_MODULE_INFO	DTS文件定义的模组信息(模组名称等)，通过该命令上 传给camera_engine与HAL，用于设备匹配。
RKMODULE_AWB_CFG	模组OTP信息使能情况下，camera_engine通过该命令传递典型模组AWB标定值，CIS驱动负责与当前模组AWB标定值比较后，生成R/B Gain值设置到CIS MWB模块中；
RKMODULE_LSC_CFG	模组OTP信息使能情况下，camera_engine通过该命令控制LSC标定值生效使能；
PREISP_CMD_SET_HDRAE_EXP	HDR曝光设置
RKMODULE_SET_HDR_CFG	设置HDR模式，可实现normal和hdr切换，需要驱动适配hdr和normal 2组配置信息
RKMODULE_GET_HDR_CFG	获取当前HDR模式
RKMODULE_SET_CONVERSION_GAIN	设置线性模式的conversion gain，如imx347、 os04a10 sensor带有conversion gain的功能，高转换 的conversion gain可以在低照度下获得更好的信噪 比，如sensor不支持conversion gain，可不实现
RKMODULE_SET_QUICK_STREAM	仅配置sensor是stream on/off寄存器，用于异常复 位。RK3588支持多摄同步，用于统一开启数据流
RKMODULE_GET_CHANNEL_INFO	RK3588新增获取通道信息，默认vicap的id0~id3对应vc0~vc3，如果有特殊需求可通过这个命令配置通道信息，如SPD/EBD数据采集。3588以前芯片仍通过get_fmt获取通道信息。
RKMODULE_GET_SYNC_MODE、RKMODULE_SET_SYNC_MODE	RK3588新增同步模式获取及配置

2. v4l2_subdev_video_ops成员回调函数实现

该结构体成员主要用于video控制的回调，具体需要实现如下成员函数：

成员名称	描述
.s_stream	开关数据流的函数，对于mipi clk是continuous的模式，必须在这个回调函 数内开启数据流，若提前开数据流，会识别不到MIPI LP状态
.g_frame_interval	获取帧间隔参数（帧率）
.g_mbus_config	linux 5.10及之前版本，linux6.1的版本挪到v4l2_subdev_pad_ops中； 获取总线配置，对于MIPI接口，sensor驱动内若支持不同lane数配置或者支 持HDR,通过这个接口返回当前sensor工作模式下的MIPI配置

3. v4l2_subdev_pad_ops成员回调函数实现

该结构体成员函数用户控制子设备，主要实现如下：

成员名称	描述
.enum_mbus_code	枚举当前CIS驱动支持数据格式
.enum_frame_size	枚举当前CIS驱动支持分辨率
.enum_frame_interval	枚举sensor支持的帧间隔，包含分辨率
.get_fmt	RKISP driver通过该回调获取CIS输出的数据格式，务必实现；针对 Bayer raw sensor、SOC yuv sensor、BW raw sensor输出的数据类 型
.set_fmt	设置CIS驱动输出数据格式以及分辨率，务必实现
.get_selection	配置裁剪参数，isp输入的宽度要求16对齐，高度8对齐。
.get_mbus_config	linux6.1之后版本修改。获取总线配置，对于MIPI接口，sensor驱动内若支持不同lane数配置或者支 持HDR,通过这个接口返回当前sensor工作模式下的MIPI配置

4. v4l2_ctrl_ops 成员回调实现

该结构体成员函数用于用于实现一些控制，主要包括如下：

成员名称	描述
.s_ctrl	RKISP driver、camera_engine通过设置不同的命令来实现CIS 曝光增益控制

主要实现的控制类型如下所示，5.1.6还会详细介绍

成员名称	描述
V4L2_CID_VBLANK	调整VBLANK，进而调整frame rate、Exposure time max
V4L2_CID_EXPOSURE	设置曝光时间，单位：曝光行数
V4L2_CID_ANALOGUE_GAIN	设置曝光增益，实际为total gain = analog gain*digital gain; 单位：增益寄存器值
V4L2_CID_HFLIP	设置水平镜像，isp不带镜像功能，如果需要镜像，使用sensor的镜像机制
V4L2_CID_VFLIP	设置垂直镜像，isp不带镜像功能，如果需要镜像，使用sensor的镜像机制
V4L2_CID_TEST_PATTERN	实现test pattern，非必须，可用于调试，根据需要实现

另外，驱动还需要实现一些获取类的控制ID，具体有，后续在5.1.7在详细举例介绍

成员名称	描述
V4L2_CID_LINK_FREQ	目前RKISP driver根据该命令获取MIPI总线频率
V4L2_CID_EXPOSURE	针对MIPI总线：pixel_rate = link_freq * 2 * nr_of_lanes / bits_per_sample

RK camera_engine会通过以上命令获取必要信息来计算曝光，其中涉及的公式如下：

公式
line_time = HTS / PIXEL_RATE;
PIXEL_RATE = HTS * VTS * FPS
HTS = sensor_width_out + HBLANK;
VTS = sensor_height_out + VBLANK;

5.1.3 初始化配置序列

sensor需要配置初始化序列才能正常工作，驱动需要在开流的时候实现初始化序列的配置。

```
//定义初始化寄存器配置
static const struct regval ov16880_global_regs[] = {
......

//不同的模式指定不同的序列
static const struct ov16880_mode supported_modes[] = {
    {
        .width = 4672,
        .height = 3504,
        .max_fps = {
            .numerator = 10000,
            .denominator = 300000,
        },
        .exp_def = 0x0ee8,
        .hts_def = 0x13a0,
        .vts_def = 0x0ef0,
        .bpp = 10,
        .reg_list = ov16880_4672x3504_30fps_regs,
        .link_freq_idx = 0,
        .hdr_mode = NO_HDR,
        .vc[PAD0] = 0,
    },
};
//实现寄存器读写函数
ov16880_write_reg
ov16880_write_array
ov16880_read_reg

//开关数据流，开流时将初始化配置写入，并打开数据流输出，关流时关闭数据流输出
static int __ov16880_start_stream(struct ov16880 *ov16880)
{
    int ret;

    ret = ov16880_write_array(ov16880->client, ov16880_global_regs);
    if (ret)
        return ret;

    ret = ov16880_write_array(ov16880->client, ov16880->cur_mode->reg_list);
    if (ret)
        return ret;
```

```
    /* In case these controls are set before streaming */
    mutex_unlock(&ov16880->mutex);
    ret = v4l2_ctrl_handler_setup(&ov16880->ctrl_handler);
    mutex_lock(&ov16880->mutex);
    if (ret)
        return ret;

    return ov16880_write_reg(ov16880->client,
                OV16880_REG_CTRL_MODE,
                OV16880_REG_VALUE_08BIT,
                OV16880_MODE_STREAMING);
}

static int __ov16880_stop_stream(struct ov16880 *ov16880)
{
    return ov16880_write_reg(ov16880->client,
                OV16880_REG_CTRL_MODE,
                OV16880_REG_VALUE_08BIT,
                OV16880_MODE_SW_STANDBY);
}

//在框架的开关流接口中调用
static int ov16880_s_stream(struct v4l2_subdev *sd, int on)
{
......
    if (on) {
        __ov16880_start_stream(ov16880);
    } else {
        __ov16880_stop_stream(ov16880);
    }
......
}
```

上述supported_modes的参数配置中，需要注意如下几个参数的配置，vts_def和hts_def需要按照寄存器序列中vts和hts寄存器配置的默认大小去设置。exp_def可以查看datasheet，如没看到，可以设置略小于vts_def，一般可以设置vts-8。

```
    .exp_def = 0x0ee8,
    .hts_def = 0x13a0,
    .vts_def = 0x0ef0,
```

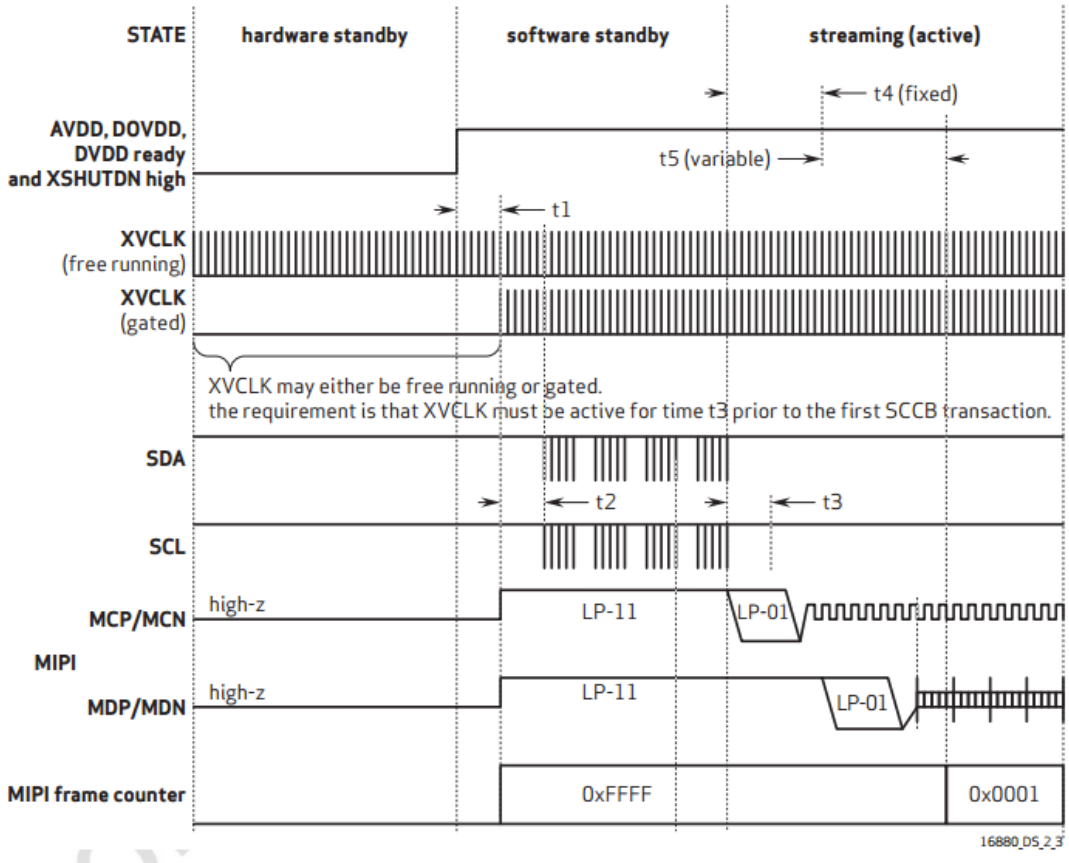
注：vts在有的sensor的描述中也称为frame length，hts也称为line length。

5.1.4 上电时序

sensor的上电通常包括：供电（avdd/dvdd/dovdd）、工作时钟（24M/27M）、reset、powerdown控制。sensor的上电会影响主控与sensor的i2c通信，有的sensor对上电时序有严格的要求。驱动需要根据规格书的上电时序要求进行配置。多数sensor对三路供电的要求没有十分严格，驱动可以按照regulator统一控制上电的方式，clk可以采用外部晶振也可以采用主控输出的方式，一般为了控制都是复用主控的IO，由主控输出。具体参考如下：

以ov16880为例，如下是其datasheet描述的上电时序：

figure 2-3 power up sequence



从上述上电时序描述图可知，该sensor对avdd、dovdd、dvdd以及reset的控制没有严格的时序要求，在上电、控制reset的t1+t2的延时之后才可以启动i2c读写操作，在clk启动之后的t2时间才可以开启i2c读写操作。对时间的描述具体如下：

table 2-3 power up sequence timing constraints

constraint	label	min	max	unit
XSHUTDN rising – system ready	t1	5		ms
minimum number of XVCLK cycles prior to the first SCCB transaction	t2	8192		XVCLK cycles
PLL start up/lock time	t3		0.2	ms
entering streaming mode – first frame start sequence (fixed part)	t4		10	ms
entering streaming mode – first frame start sequence (variable part)	t5	delay is exposure time value		lines

最后依据上述时序描述实现的上电时序配置如下，如果是对供电通路有严格时序要求的，请严格按照datasheet的描述进行实现。

```
static int __ov16880_power_on(struct ov16880 *ov16880)
{
    int ret;
    u32 delay_us;
    struct device *dev = &ov16880->client->dev;

    if (!IS_ERR(ov16880->power_gpio))
        gpiod_set_value_cansleep(ov16880->power_gpio, 1);

    usleep_range(1000, 2000);
```

```
    if (!IS_ERR_OR_NULL(ov16880->pins_default)) {
        ret = pinctrl_select_state(ov16880->pinctrl,
                       ov16880->pins_default);
        if (ret < 0)
            dev_err(dev, "could not set pins\n");
    }
    ret = clk_set_rate(ov16880->xvclk, OV16880_XVCLK_FREQ);
    if (ret < 0)
        dev_warn(dev, "Failed to set xvclk rate (24MHz)\n");
    if (clk_get_rate(ov16880->xvclk) != OV16880_XVCLK_FREQ)
        dev_warn(dev, "xvclk mismatched, modes are based on 24MHz\n");
    ret = clk_prepare_enable(ov16880->xvclk);
    if (ret < 0) {
        dev_err(dev, "Failed to enable xvclk\n");
        return ret;
    }
    if (!IS_ERR(ov16880->reset_gpio))
        gpiod_set_value_cansleep(ov16880->reset_gpio, 0);

    ret = regulator_bulk_enable(OV16880_NUM_SUPPLIES, ov16880->supplies);
    if (ret < 0) {
        dev_err(dev, "Failed to enable regulators\n");
        goto disable_clk;
    }

    if (!IS_ERR(ov16880->reset_gpio))
        gpiod_set_value_cansleep(ov16880->reset_gpio, 1);

    usleep_range(5000, 6000);
    if (!IS_ERR(ov16880->pwdn_gpio))
        gpiod_set_value_cansleep(ov16880->pwdn_gpio, 1);

    /* 8192 cycles prior to first SCCB transaction */
    delay_us = ov16880_cal_delay(8192);
    usleep_range(delay_us * 2, delay_us * 3);

    return 0;

disable_clk:
    clk_disable_unprepare(ov16880->xvclk);

    return ret;
}
```

需要注意的是reset、powerdown的pin的极性控制，一般datasheet会给出对应的有效极性，驱动代码中最终设置的值为1并不代表该gpio输出高电平状态，输出是否为高低电平，需要代码控制逻辑与DTS配置的有效极性同时决定。按照如下规律：

> 若DTS配置的有效极性为高有效，则：
>
> 驱动设置value为1输出高电平，驱动设置value为0输出低电平
>
> 若DTS配置的有效极性为低有效，则：
>
> 驱动设置value为0输出高电平，驱动设置value为1输出低电平

sensor的datasheet对reset、powdn的极性描述一般如下，需要注意的是datasheet描述低电平有效意味着低电平会处于reset或者powerdown的状态，所以需要sensor正常工作，这两个pin需要给高电平。

66	DOGND	ground	ground for I/O circuit
67	PWDNB	input	power down (active low with pull up resistor)
68	XSHUTDN	input	reset and power down (active low with pull down resistor)
69	DOVDD	power	power for I/O circuit

框架的上下电函数在.s_power函数接口中实现，需要在s_power函数接口中调用对应的上下电函数，由于我们的sensor大多数采用pm的管理机制，因此s_power函数需要对pm的引用计数进行控制。

5.1.5 下电时序

一般的sensor对下电时序都没有严格的要求，下电仅需要将对应的供电关闭、clk关闭，对应的gpio拉低即可。具体参考如下：

```
static void __ov16880_power_off(struct ov16880 *ov16880)
{
    int ret;
    struct device *dev = &ov16880->client->dev;

    if (!IS_ERR(ov16880->pwdn_gpio))
        gpiod_set_value_cansleep(ov16880->pwdn_gpio, 0);
    clk_disable_unprepare(ov16880->xvclk);
    if (!IS_ERR(ov16880->reset_gpio))
        gpiod_set_value_cansleep(ov16880->reset_gpio, 0);

    if (!IS_ERR_OR_NULL(ov16880->pins_sleep)) {
        ret = pinctrl_select_state(ov16880->pinctrl,
                       ov16880->pins_sleep);
        if (ret < 0)
            dev_dbg(dev, "could not set pins\n");
    }
    if (!IS_ERR(ov16880->power_gpio))
        gpiod_set_value_cansleep(ov16880->power_gpio, 0);

    regulator_bulk_disable(OV16880_NUM_SUPPLIES, ov16880->supplies);
}
```

5.1.6 曝光增益等控制实现

此部分仅针对CMOS sensor 输出Bayer RAW数据情况，输出YUYV格式的CMOS Sensor默认是Sensor内部已经做好曝光和增益控制。 输出Bayer RAW的sensor，主控的ISP需要通过3A算法计算曝光增益，并将对应的参数设置到sensor中用以控制sensor获取最佳的图像效果。故对于RAW sensor中，曝光增益的控制是至关重要的，若驱动对该处实现的有误，则可能影响图像的效果。这部分的具体在set_ctrl接口中实现，这部分主要是对曝光、gain、以及vblank（VTS）进行控制，需要按照sensor的手册进行控制，以下以ov16880为例具体介绍如何实现这部分。

1. 曝光寄存器控制

在datasheet中找到曝光的寄存器描述，可以看出曝光占用三个寄存器，其中0x3500寄存器的bit0-3是曝光的高位，其他bit没有使用，因此控制寄存器的时候可以直接写入这三个寄存器。

table 6-26 MEC/MGC control registers (sheet 1 of 2)

address	register name	default value	R/W	description
0x3500	EXPO LONG PK	0x00	RW	Bit[7:4]: Not used Bit[3:0]: Long exposure[19:16]
0x3501	MEC LONG EXPO	0x02	RW	Long Exposure Bit[7:0]: Long exposure[15:8]
0x3502	MEC LONG EXPO	0x00	RW	Long Exposure Bit[7:0]: Long exposure[7:0]

在set_ctrl中实现如下：

```
//定义寄存器地址
#define OV16880_REG_EXPOSURE_H      0x3500
#define OV16880_REG_EXPOSURE_M      0x3501
#define OV16880_REG_EXPOSURE_L      0x3502
#define OV16880_EXPOSURE_MIN        8
#define OV16880_EXPOSURE_STEP       1
#define OV16880_VTS_MAX         0x7fff
//set_ctrl中实现
......
    case V4L2_CID_EXPOSURE:
        /* 4 least significant bits of expsoure are fractional part */
        ret = ov16880_write_reg(ov16880->client,
                    OV16880_REG_EXPOSURE_H,
                    OV16880_REG_VALUE_24BIT,
                    ctrl->val & 0xfffff);
        dev_dbg(&client->dev, "set exposure 0x%x\n",
            ctrl->val);
        break;
......
```

有一些sensor并非如此按照按照bit顺序写入曝光寄存器，则需要驱动进行移位控制，例如ov13855的曝光寄存器描述，低4为是小数位，但是传感器不支持小数，所以必须设置为0，因此在曝光控制的时候就不能直接写入寄存器，直接写入会导致实际的曝光写到小数位上面，从而导致图像异常。ov13855在控制的时候需要移位后再写入。

address	register name	value	R/W	description
0x3501	LONG EXPO	0x02	RW	Long Exposure Bit[7:0]: Long exposure[15:8]
0x3502	LONG EXPO	0x00	RW	Long Exposure Bit[7:0]: Long exposure[7:0] Low 4 bits are fraction bits This sensor does not support fraction exposure, so low 4 bits cannot be used and must be set to 0.

```
    case V4L2_CID_EXPOSURE:
        /* 4 least significant bits of expsoure are fractional part */
        ret = ov13855_write_reg(ov13855->client,
                    OV13855_REG_EXPOSURE,
                    OV13855_REG_VALUE_24BIT,
                    ctrl->val << 4);
        break;
```

2. 增益寄存器控制

增益又有数字增益与模拟增益，一般只使用模拟增益，如果模拟增益实在不够，才会使用数字增益。因为一般数字增益引入噪声比模拟增益大。最终总增益为数字增益与模拟增益的乘积，即total gain = again * dgain。但是在我们的驱动框架中只需要实现V4L2_CID_ANALOGUE_GAIN一个控制，将again与dgain放在一起控制，这里我们以ov16880（控制again）与ov16a10（控制again+dgain）为例分别进行介绍。

ov16880对again的描述如下，没有什么特殊的描述，在驱动中直接控制即可。

0x350A	MEC LONG GAIN	0x00	RW	Long Gain Bit[7:3]: Not used Bit[2:0]: Long gain[10:8]
0x350B	MEC LONG GAIN	0x10	RW	Long Gain Bit[7:0]: Long gain[7:0]

```
//定义寄存器地址
#define OV16880_AGAIN_REG_H     0x350A
#define OV16880_AGAIN_REG_L     0x350B
//定义again的最大最小默认值等
#define OV16880_GAIN_MIN        0x10
#define OV16880_GAIN_MAX        0x7FF
#define OV16880_GAIN_STEP       1
#define OV16880_GAIN_DEFAULT    0x20
......
    case V4L2_CID_ANALOGUE_GAIN:
        ret = ov16880_write_reg(ov16880->client,
                    OV16880_AGAIN_REG_H,
                    OV16880_REG_VALUE_16BIT,
                    ctrl->val & 0x7ff);
        dev_dbg(&client->dev, "set analog gain value 0x%x\n", ctrl->val);
        break;
......
```

ov16a10对gain的寄存器描述如下：0x3509寄存器的bit1-7为again的高位，0x3508的bit0-6为again的低位，因此在again控制的时候需要左移一位。同样的0x350c的bit6-7才是dgain使用，因此控制dgain的时候需要左移6位。

0x3508	GAIN COARSE	0x01	RW	Bit[7]: Not used Bit[6:0]: gain_coarse[6:0]
0x3509	CTRL 09	0x00	RW	Bit[7:1]: gain_fine Bit[0]: Not used
0x350A	NON RISC A	0x01	RW	Bit[7:4]: Reserved Bit[3:0]: dig_gain_coarse_b[3:0]
0x350B	NON RISC B	0x00	RW	Bit[7:0]: dig_gain_fine_b[9:2]
0x350C	NON RISC C	0x00	RW	Bit[7:6]: dig_gain_fine_b[1:0] Bit[5:0]: Reserved

ov16a10 again 为0x80是1x，最大agian为15.5x，即最大为1984，当未使用dgain时即dgain为1x时，写入值位1024。此部分可与sensor厂家进行确认，有的sensor的厂家提供的文档没有详细说明。故ov16a10按照如下方式实现gain的控制，超过agian时，需要计算dgain的大小并设置。

```
#define OV16A10_REG_AGAIN_H     0x3508
#define OV16A10_REG_AGAIN_L     0x3509
#define OV16A10_REG_DAGAIN_H_B      0x350A
#define OV16A10_REG_DAGAIN_M_B      0x350B
```

```
#define OV16A10_REG_DAGAIN_L_B      0x350C
#define OV16A10_GAIN_MIN        0x80
#define OV16A10_GAIN_MAX        0x3df61
#define OV16A10_GAIN_STEP       1
#define OV16A10_GAIN_DEFAULT        0x80
......
    case V4L2_CID_ANALOGUE_GAIN:
        if (ctrl->val > 1984) {// >15.5x
            dgain = ctrl->val * 10 / 155;
            again = 1984;
        } else {
            dgain = 1024;
            again = ctrl->val;
        }
        ret |= ov16a10_write_reg(ov16a10->client,
                     OV16A10_REG_AGAIN_H,
                     OV16A10_REG_VALUE_16BIT,
                     (again << 1) & 0x7ffe);
        ret |= ov16a10_write_reg(ov16a10->client,
                     OV16A10_REG_DAGAIN_H_B,
                     OV16A10_REG_VALUE_24BIT,
                     (dgain << 6) & 0xfffc0);

        dev_dbg(&client->dev, "set gain 0x%x set analog gain 0x%x digital gain
0x%x\n",
            ctrl->val, again, dgain);
        break;
......
```

3. vblank控制

V4L2_CID_VBLANK上层一般会计算vblank后下发，vts = vblank + height，vts的控制一般少有特殊的描述，直接将计算的vts写入寄存器即可，如下所示：

```
#define OV16880_REG_VTS_H       0x380e
#define OV16880_REG_VTS_L       0x380f
......
    case V4L2_CID_VBLANK:
        ret = ov16880_write_reg(ov16880->client,
                    OV16880_REG_VTS_H,
                    OV16880_REG_VALUE_16BIT,
                    ctrl->val + ov16880->cur_mode->height);
        break;
......
```

4. 其他控制

看一些已经调试过sensor驱动我们可以看到都还有带有pattern、vfilp、hflip的控制，这部分的控制不是必须实现的，调试的时候可以不用实现。pattern即出测试图，vfilp即上下镜像设置，hflip即左右镜像设置。这部分实现比较简单，直接根据寄存器的控制即可，若没有特殊需求，也可以直接不实现（建议不用实现，有的sensor在flip之后，可能会导致byer排列变化，还需要增加offset的控制，可能会导致图像异常或者效果调试的时候blc的异常）。

5.1.7 MIPI时钟与像素时钟配置

如果是MIPI接口的sensor，需要注意MIPI时钟的配置，可以先跟sensor输出端确定输出的MIPI时钟大小，如无法确定时钟大小，建议直接使用示波器进行测量。时钟配置错误，可能会导致出现MIPI报错从而导致出图异常。

MIPI时钟即对应驱动的link-freq的控制参数，具体控制与实现流程大体如下：

```
//定义link-freq控制
ov16880->link_freq = v4l2_ctrl_new_int_menu(handler, NULL, V4L2_CID_LINK_FREQ, 0,
0, link_freq_items);

//定义link-freq的数组
#define OV16880_LINK_FREQ_720MHZ    720000000U
......
static const s64 link_freq_items[] = {
    OV16880_LINK_FREQ_720MHZ,
};

//不同的mode配置设置不同的index，根据上述的items数组索引
static const struct ov16880_mode supported_modes[] = {
    {
        .width = 4672,
        .height = 3504,
......
        .reg_list = ov16880_4672x3504_30fps_regs,
        .link_freq_idx = 0,
......
    },
};

//在set_fmt接口中根据不同的index进行控制
static int ov16880_set_fmt(struct v4l2_subdev *sd,
               struct v4l2_subdev_state *sd_state,
              struct v4l2_subdev_format *fmt)
{
......
        __v4l2_ctrl_s_ctrl(ov16880->link_freq, mode->link_freq_idx);
......
}
```

像素时钟一般与MIPI时钟对应有比例的关系，我们可以根据MIPI时钟的大小控制像素时钟。参考如下：

```
/* pixel rate = link frequency * 2 * lanes / BITS_PER_SAMPLE */
#define OV16880_PIXEL_RATE      (OV16880_LINK_FREQ_720MHZ * 2LL * 4LL / 10LL)

//定义像素时钟
dst_pixel_rate = (u32)link_freq_items[mode->link_freq_idx] / mode->bpp * 2 *
lane_num;
ov16880->pixel_rate = v4l2_ctrl_new_std(handler, NULL, V4L2_CID_PIXEL_RATE, 0,
OV16880_PIXEL_RATE, 1, dst_pixel_rate);

//控制像素时钟
static int ov16880_set_fmt(struct v4l2_subdev *sd,
               struct v4l2_subdev_state *sd_state,
```

```
                struct v4l2_subdev_format *fmt)
{
......
        __v4l2_ctrl_s_ctrl_int64(ov16880->pixel_rate, pixel_rate);
......
}
```

5.1.8 HDR模式调试

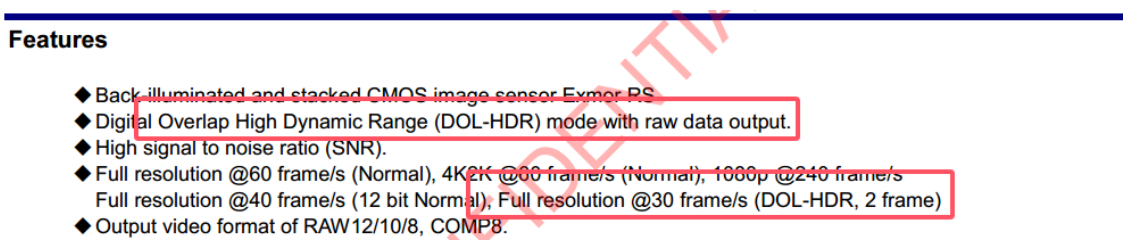
在5.1.2中的ioctl列表中，已经简单介绍过HDR相关配置的控制ID，这里以一个具体的sensor实现为例进行介绍。需要实现HDR模式，首先需要确认sensor是否可以支持HDR，以及对应支持的HDR的类型是什么，明确与主控所支持的HDR的类型是否一致。RK的平台支持mipi sensor虚拟通道输出2帧或者3帧的模式，ISP端进行多帧合成，具体的规格可以参考对应的芯片文档，以下是主要主控芯片的支持情况。

芯片名称	HDR	Support	Not Support
RK3588/RK3588S	2/3-Frame Merge into High-Dynamic Range	Sequential/Framebase HDR Staggered/DOL HDR DCG HDR	SME HDR BME HDR QBC HDR Lateral overflow HDR Split-diode pixel HDR
RK3576	2-Frame Merge into High-Dynamic Range	Sequential/Framebase HDR Staggered/DOL HDR DCG HDR	SME HDR BME HDR QBC HDR Lateral overflow HDR Split-diode pixel HDR
RK3562	2-Frame Merge into High-Dynamic Range	Sequential/Framebase HDR Staggered/DOL HDR DCG HDR	SME HDR BME HDR QBC HDR Lateral overflow HDR Split-diode pixel HDR
RK3568/RK3566	2-Frame Merge into High-Dynamic Range	Sequential/Framebase HDR Staggered/DOL HDR DCG HDR	SME HDR BME HDR QBC HDR Lateral overflow HDR Split-diode pixel HDR
RK3572	2-Frame Merge into High-Dynamic Range	Sequential/Framebase HDR Staggered/DOL HDR DCG HDR	SME HDR BME HDR QBC HDR Lateral overflow HDR Split-diode pixel HDR

HDR模式的调试与上述线性模式的调试流程类似，重复部分这里不再赘述，以下以IMX577为例介绍如何实现HDR模式。

1. 确认sensor支持HDR

可以根据sensor的datasheet确认HDR的支持情况，如下图是IMX577对支持HDR模式的描述。



2. 获取HDR配置序列及HDR控制手册

HDR模式的寄存器配置与线性模式差别较大，建议直接与原厂沟通获取HDR的配置序列，另外也需要获取到HDR的详细描述文档，方便驱动实现。

3. 驱动添加HDR模式

驱动支持模式中需要添加HDR模式的支持，参考如下所示在support模式中增加HDR模式，此处需要注意的是link-freq有可能与线性模式不同需要增加，vc需要指定长短帧的输出pad。

```
@@ -656,6 +941,26 @@ static const struct imx577_mode supported_modes[] = {
         .link_freq_idx = 1,
         .vc[PAD0] = V4L2_MBUS_CSI2_CHANNEL_0,
     },
+    {
+        .width = 4056,
+        .height = 3040,
+        .max_fps = {
+            .numerator = 10000,
+            .denominator = 250000,
+        },
+        .exp_def = 0x0c10,
+        .hts_def = 0x11a0,
+        .vts_def = 0x0e8a,
+        .bpp = 10,
+        .bus_fmt = MEDIA_BUS_FMT_SRGGB10_1X10,
+        .reg_list = imx577_hdr2_10bit_4056x3040_30fps_regs,
+        .link_freq_idx = 0,
+        .hdr_mode = HDR_X2,
+        .vc[PAD0] = V4L2_MBUS_CSI2_CHANNEL_1,
+        .vc[PAD1] = V4L2_MBUS_CSI2_CHANNEL_0,//L->csi wr0
+        .vc[PAD2] = V4L2_MBUS_CSI2_CHANNEL_1,
+        .vc[PAD3] = V4L2_MBUS_CSI2_CHANNEL_1,//M->csi wr2
+    },
```

4. 曝光增益控制

线性模式exposure、gain通过标准v4l2命令实现，当切换成HDR模式时，需要通过ioctl命令PREISP_CMD_SET_HDRAE_EXP传递参数，标准v4l2命令中的V4L2_CID_EXPOSURE、V4L2_CID_ANALOGUE_GAIN不使用。PREISP_CMD_SET_HDRAE_EXP实现时需要能够缓存参数，在start_stream写完初始化数组后，需要将初始曝光重新写进sensor，覆盖掉初始化数组里面的默认值，这样数据流输出的第一帧亮度才能和曝光参数匹配，才能尽快完成ae收敛。需要注意的是初始化数组里面不能配置开启数据流的寄存器，需要在start_stream返回前配置，同样是为了保证输出的第一帧亮度和初始化曝光参数匹配。

此处实现的时候需要注意的是，HDR的控制寄存器一般与线性模式有所区别，所以需要与sensor端确认清楚HDR模式下的控制寄存器。同时也要确认HDR模式下对曝光增益的限制范围，均需要在驱动中实现，如下参考IMX577驱动的实现：

IMX577 HDR手册描述对曝光限制：

Exposure time settings for 2 frame DOL-HDR must be set with below restrictions.

No.	Item	Restriction
1	Range of DOL_CIT_1ST	2 ≤ DOL_CIT_1ST ≤ "FLL - (2 × CIT_MARGIN) - 2"
2	Range of DOL_CIT_2ND	2 ≤ DOL_CIT_2ND ≤ "FLL - BRL - CIT_MARGIN"
3	Maximum exposure time	DOL_CIT_1ST + DOL_CIT_2ND + (2 × CIT_MARGIN) ≤ FLL
4	Range of DOL_FRAME_OFFSET_2ND	DOL_CIT_2ND + CIT_MARGIN ≤ DOL_FRAME_OFFSET_2ND ≤ "FLL - BRL"
5	Value of DOL_CIT_1ST, DOL_CIT_2ND and DOL_FRAME_OFFSET_2ND	DOL_CIT_1ST, DOL_CIT_2ND and DOL_FRAME_OFFSET_2ND must be set to even value.

In general
DOL_FRAME_OFFSET_2ND = DOL_CIT_1ST + CIT_MARGIN.
CIT_MARGIN = 22 [lines] (Fixed)

IMX577根据上述曝光限制，对曝光增益进行控制驱动实现：

```
//imx577对应的HDR模式控制寄存器
#define IMX577_LF_GAIN_REG_H        0x00f0
#define IMX577_LF_GAIN_REG_L        0x00f1
#define IMX577_SEF1_GAIN_REG_H      0x00f2
#define IMX577_SEF1_GAIN_REG_L      0x00f3
#define IMX577_SEF2_GAIN_REG_H      0x00f4
#define IMX577_SEF2_GAIN_REG_L      0x00f5

#define IMX577_LF_DGAIN_REG_H       0x00f6
#define IMX577_LF_DGAIN_REG_L       0x00f7
#define IMX577_SEF1_DGAIN_REG_H     0x00f8
#define IMX577_SEF1_DGAIN_REG_L     0x00f9
#define IMX577_SEF2_DGAIN_REG_H     0x00fa
#define IMX577_SEF2_DGAIN_REG_L     0x00fb

#define IMX577_LF_EXPO_REG_H        0x00ea
#define IMX577_LF_EXPO_REG_L        0x00eb
#define IMX577_SEF1_EXPO_REG_H      0x00ec
#define IMX577_SEF1_EXPO_REG_L      0x00ed
#define IMX577_SEF2_EXPO_REG_H      0x00ee
#define IMX577_SEF2_EXPO_REG_L      0x00ef

#define IMX577_RHS1_REG_H       0x00e6
#define IMX577_RHS1_REG_L       0x00e7
#define IMX577_RHS2_REG_H       0x00e8
#define IMX577_RHS2_REG_L       0x00e9
//HDR模式ae实现函数
static int imx577_set_hdrae(struct imx577 *imx577,
                struct preisp_hdrae_exp_s *ae)
{
    struct i2c_client *client = imx577->client;
    u32 l_exp_time, m_exp_time, s_exp_time;
    u32 l_a_gain, m_a_gain, s_a_gain;
    u32 l_d_gain, s_d_gain;
    int ret = 0;
    u32 fll, dol_cit1, dol_cit2, dol_off2;

    if (!imx577->has_init_exp && !imx577->streaming) {
        imx577->init_hdrae_exp = *ae;
        imx577->has_init_exp = true;
        dev_dbg(&imx577->client->dev, "imx577 is not streaming, save hdr ae!\n");
```

```
        return ret;
    }
    l_exp_time = ae->long_exp_reg;
    m_exp_time = ae->middle_exp_reg;
    s_exp_time = ae->short_exp_reg;
    l_a_gain = ae->long_gain_reg;
    m_a_gain = ae->middle_gain_reg;
    s_a_gain = ae->short_gain_reg;
    dev_dbg(&client->dev,
        "rev exp req: L_exp: 0x%x, 0x%x, M_exp: 0x%x, 0x%x S_exp: 0x%x, 0x%x\n",
        l_exp_time, m_exp_time, s_exp_time,
        l_a_gain, m_a_gain, s_a_gain);

    if (imx577->cur_mode->hdr_mode == HDR_X2) {
        l_a_gain = m_a_gain;
        l_exp_time = m_exp_time;
    }

    ret = imx577_write_reg(client, IMX577_GROUP_HOLD_REG,
        IMX577_REG_VALUE_08BIT, IMX577_GROUP_HOLD_START);
    /* gain effect n+1 */
    if (l_a_gain > 0x1600)
        l_a_gain = 0x1600;
    if (l_a_gain < 0x10)
        l_a_gain = 0x10;
    if (s_a_gain > 0x1600)
        s_a_gain = 0x1600;
    if (s_a_gain < 0x10)
        s_a_gain = 0x10;
    l_d_gain = l_a_gain > 0x160 ? (l_a_gain * 256 / 22 / 16) : 256;
    l_a_gain = l_a_gain > 0x160 ? 0x160 : l_a_gain;
    l_a_gain = 1024 - 1024 * 16 / l_a_gain;
    s_d_gain = s_a_gain > 0x160 ? (s_a_gain * 256 / 22 / 16) : 256;
    s_a_gain = s_a_gain > 0x160 ? 0x160 : s_a_gain;
    s_a_gain = 1024 - 1024 * 16 / s_a_gain;

    ret |= imx577_write_reg(client, IMX577_LF_GAIN_REG_H,
        IMX577_REG_VALUE_08BIT, IMX577_FETCH_GAIN_H(l_a_gain));
    ret |= imx577_write_reg(client, IMX577_LF_GAIN_REG_L,
        IMX577_REG_VALUE_08BIT, IMX577_FETCH_GAIN_L(l_a_gain));
    ret |= imx577_write_reg(client, IMX577_SEF1_GAIN_REG_H,
        IMX577_REG_VALUE_08BIT, IMX577_FETCH_GAIN_H(s_a_gain));
    ret |= imx577_write_reg(client, IMX577_SEF1_GAIN_REG_L,
        IMX577_REG_VALUE_08BIT, IMX577_FETCH_GAIN_L(s_a_gain));

    if (IMX577_DGAIN_MODE && l_d_gain > 0 && s_d_gain > 0) {
        ret |= imx577_write_reg(client, IMX577_LF_DGAIN_REG_H,
            IMX577_REG_VALUE_08BIT, IMX577_FETCH_DGAIN_H(l_d_gain));
        ret |= imx577_write_reg(client, IMX577_LF_DGAIN_REG_L,
            IMX577_REG_VALUE_08BIT, IMX577_FETCH_DGAIN_L(l_d_gain));
        ret |= imx577_write_reg(client, IMX577_SEF1_DGAIN_REG_H,
            IMX577_REG_VALUE_08BIT, IMX577_FETCH_DGAIN_H(s_d_gain));
        ret |= imx577_write_reg(client, IMX577_SEF1_DGAIN_REG_L,
            IMX577_REG_VALUE_08BIT, IMX577_FETCH_DGAIN_L(s_d_gain));
    }

    fll = imx577->cur_vts;
    dol_cit1 = l_exp_time >> 1;
```

```
    dol_cit2 = s_exp_time >> 1;

    /*dol_cit1 dol_cit2 dol_off2 should be even*/
    if (dol_cit1 < 2)
        dol_cit1 = 2;
    else if (dol_cit1 > fll - 2 * CIT_MARGIN - 2)
        dol_cit1 = fll - 2 * CIT_MARGIN - 2;
    dol_cit1 &= (~0x1);

    if (dol_cit2 < 2)
        dol_cit2 = 2;
    else if (dol_cit2 > fll - BRL_FULL - CIT_MARGIN)
        dol_cit2 = fll - BRL_FULL - CIT_MARGIN;
    dol_cit2 &= (~0x1);

    dol_off2 = (dol_cit2 + CIT_MARGIN) & (~0x1);
    if (dol_off2 < dol_cit2 + CIT_MARGIN)
        dol_off2 = (dol_cit2 + CIT_MARGIN) & (~0x1);
    else if (dol_off2 > fll - BRL_FULL)
        dol_off2 = (fll - BRL_FULL) & (~0x1);

    dev_dbg(&client->dev,
        "l_exp_time=%d,s_exp_time=%d,fll=%d,rhs1=%d,l_a_gain=%d,s_a_gain=%d\n",
        l_exp_time, s_exp_time, fll, dol_off2, l_a_gain, s_a_gain);

    ret |= imx577_write_reg(client,
        IMX577_RHS1_REG_L,
        IMX577_REG_VALUE_08BIT,
        IMX577_FETCH_RHS1_L(dol_off2));
    ret |= imx577_write_reg(client,
        IMX577_RHS1_REG_H,
        IMX577_REG_VALUE_08BIT,
        IMX577_FETCH_RHS1_H(dol_off2));

    ret |= imx577_write_reg(client,
        IMX577_SEF1_EXPO_REG_L,
        IMX577_REG_VALUE_08BIT,
        IMX577_FETCH_EXP_L(dol_cit2));
    ret |= imx577_write_reg(client,
        IMX577_SEF1_EXPO_REG_H,
        IMX577_REG_VALUE_08BIT,
        IMX577_FETCH_EXP_H(dol_cit2));
    ret |= imx577_write_reg(client,
        IMX577_LF_EXPO_REG_L,
        IMX577_REG_VALUE_08BIT,
        IMX577_FETCH_EXP_L(dol_cit1));
    ret |= imx577_write_reg(client,
        IMX577_LF_EXPO_REG_H,
        IMX577_REG_VALUE_08BIT,
        IMX577_FETCH_EXP_H(dol_cit1));

    ret |= imx577_write_reg(client, IMX577_GROUP_HOLD_REG,
        IMX577_REG_VALUE_08BIT, IMX577_GROUP_HOLD_END);

    return ret;
}
//HDR模式增加ae的ioctl
static long imx577_ioctl(struct v4l2_subdev *sd, unsigned int cmd, void *arg)
```

```
{
......
    switch (cmd) {
    case PREISP_CMD_SET_HDRAE_EXP:
        if (imx577->cur_mode->hdr_mode == HDR_X2)
            ret = imx577_set_hdrae(imx577, arg);
        break;
......
```

同时需要在原线性的s_ctrl控制函数中避免HDR模式的流程。类似如下：

```
//xxx_set_ctrl
......
    case V4L2_CID_ANALOGUE_GAIN:
        if (imx577->cur_mode->hdr_mode != NO_HDR)
            goto ctrl_end;
    case V4L2_CID_EXPOSURE:
        if (imx577->cur_mode->hdr_mode != NO_HDR)
            goto ctrl_end;
......
```

5. 与线性模式切换

sensor与控制器的HDR/LINEAR交互，通过RKMODULE_SET_HDR_CFG/RKMODULE_GET_HDR_CFG，HDR sensor必须实现，才能正常进行模式切换。

参考IMX577如下：

```
......//ioctl
    case RKMODULE_GET_HDR_CFG:
        hdr = (struct rkmodule_hdr_cfg *)arg;
        hdr->esp.mode = HDR_NORMAL_VC;
        hdr->hdr_mode = imx577->cur_mode->hdr_mode;
        break;
    case RKMODULE_SET_HDR_CFG:
        hdr = (struct rkmodule_hdr_cfg *)arg;
        w = imx577->cur_mode->width;
        h = imx577->cur_mode->height;
        for (i = 0; i < ARRAY_SIZE(supported_modes); i++) {
            if (w == supported_modes[i].width &&
                h == supported_modes[i].height &&
                supported_modes[i].hdr_mode == hdr->hdr_mode) {
                imx577->cur_mode = &supported_modes[i];
                break;
            }
        }
        if (i == ARRAY_SIZE(supported_modes)) {
            dev_err(&imx577->client->dev,
                "not find hdr mode:%d %dx%d config\n",
                hdr->hdr_mode, w, h);
            ret = -EINVAL;
        } else {
            mode = imx577->cur_mode;
            imx577->cur_vts = mode->vts_def;
            w = mode->hts_def - mode->width;
            h = mode->vts_def - mode->height;
```

```
            mutex_lock(&imx577->mutex);
            __v4l2_ctrl_modify_range(imx577->hblank, w, w, 1, w);
            __v4l2_ctrl_modify_range(imx577->vblank, h,
                IMX577_VTS_MAX - mode->height,
                1, h);
            __v4l2_ctrl_s_ctrl(imx577->link_freq, mode->link_freq_idx);
            dst_pixel_rate = (u32)link_freq_items[mode->link_freq_idx] /
                    mode->bpp * 2 * lane_num;
            __v4l2_ctrl_s_ctrl_int64(imx577->pixel_rate,
                         dst_pixel_rate);
            mutex_unlock(&imx577->mutex);
        }
        break;
......
```

6. group寄存器使用

在上述实现HDR的控制流程中，我们使用了sensor的group的寄存器，这里介绍一下这个寄存器的作用，该寄存器可让写寄存器的同时生效，避免长帧的寄存器写完，短帧还没开始写，长帧就已经生效了。一般在控制较多寄存器的场景下需要使用。

7. 驱动其他接口的增加

.enum_frame_interval：该接口最后的reverse需要上报对应的HDR mode。

```
@@ -1329,6 +1910,7 @@ static int imx577_enum_frame_interval(struct v4l2_subdev
*sd,
        fie->width = supported_modes[fie->index].width;
        fie->height = supported_modes[fie->index].height;
        fie->interval = supported_modes[fie->index].max_fps;
+       fie->reserved[0] = supported_modes[fie->index].hdr_mode;

        return 0;
 }
```

.g_mbus_config：配置使用的MIPI通道数量。

```
@@ -1337,12 +1919,15 @@ static int imx577_g_mbus_config(struct v4l2_subdev *sd,
unsigned int pad,
                                struct v4l2_mbus_config *config)
 {
        struct imx577 *imx577 = to_imx577(sd);
+       const struct imx577_mode *mode = imx577->cur_mode;
        u32 lane_num = imx577->bus_cfg.bus.mipi_csi2.num_data_lanes;
        u32 val = 0;

        val = 1 << (lane_num - 1) |
                V4L2_MBUS_CSI2_CHANNEL_0 |
                V4L2_MBUS_CSI2_CONTINUOUS_CLOCK;
+       if (mode->hdr_mode != NO_HDR)
+               val |= V4L2_MBUS_CSI2_CHANNEL_1;

        config->type = V4L2_MBUS_CSI2_DPHY;
        config->flags = val;
```

8. dts配置与xml配置

dts与xml需要增加对应的配置，在后续dts配置与xml配置的章节会再介绍。

5.1.9 PDAF添加

有的sensor可以支持PDAF，此章节简要介绍sensor驱动如何增加PDAF的支持。

1. 更新配置序列支持输出PD像素

PDAF需要sensor支持输出PD像素，建议采用虚拟通道的方式，vc0输出raw图，vc1输出pd像素给到主控，此处需要sensor端更新配置序列。

2. 定义PD像素数据格式与大小

PD像素通道我们需要单独传递对应的数据格式与大小，需要确认PD像素是如何输出的，分辨率是多大，需要在驱动中进行设置，参考如下代码实现：

```
//定义数据结构
struct other_data {
    u32 width;
    u32 height;
    u32 bus_fmt;
    u32 data_type;
    u32 data_bit;
};
//设置对应分辨率其输出的pd通道的大小与数据格式
static const struct other_data ov16a10_4608x3456_spd = {
    .width = 1152,
    .height = 432,
    .bus_fmt = MEDIA_BUS_FMT_SPD_2X8,
    .data_type = 0x2b,
    .data_bit = 10,
};
//在对应的模式中引用
static const struct ov16a10_mode supported_modes[] = {
    {
        .bus_fmt = OV16A10_MEDIA_BUS_FMT,
        .width = 4608,
        .height = 3456,
        .max_fps = {
            .numerator = 10000,
            .denominator = 300000,
        },
        .exp_def = 0x0f48,
        .hts_def = 0x0352 * 6,
        .vts_def = 0x0f50,
        .bpp = 10,
        .reg_list = ov16a10_4608x3456_30fps_regs,
        .link_freq_idx = 0,
        .hdr_mode = NO_HDR,
        .spd = &ov16a10_4608x3456_spd,
        .vc[PAD0] = V4L2_MBUS_CSI2_CHANNEL_0,
    },
};
```

3. link-freq更新

同HDR模式，如果PD输出的序列，MIPI输出CLK有变化，link-freq也需要对应的改变

4. 获取通道信息

PD像素通过VC1输出，raw数据通过VC0输出，主控ISP需要通过ioctl获取通道信息，具体实现如下：

```
static int ov16a10_get_channel_info(struct ov16a10 *ov16a10, struct
rkmodule_channel_info                                    *ch_info)
{
    const struct ov16a10_mode *mode = ov16a10->cur_mode;

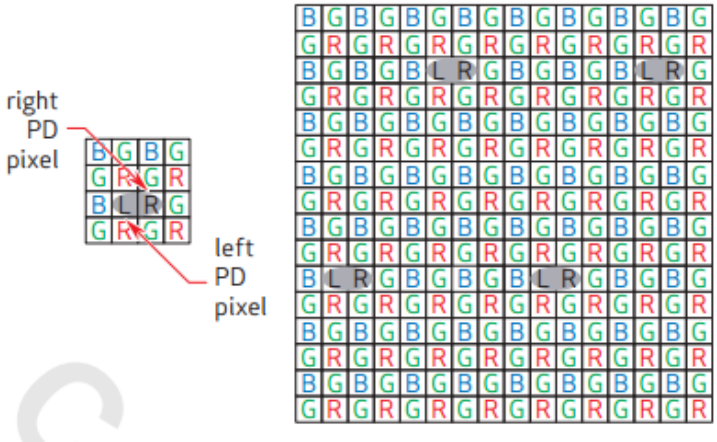
    if (ch_info->index < PAD0 || ch_info->index >= PAD_MAX)
        return -EINVAL;

    if (ch_info->index == ov16a10->spd_id && mode->spd) {
        ch_info->vc = V4L2_MBUS_CSI2_CHANNEL_1;
        ch_info->width = mode->spd->width;
        ch_info->height = mode->spd->height;
        ch_info->bus_fmt = mode->spd->bus_fmt;
        ch_info->data_type = mode->spd->data_type;
        ch_info->data_bit = mode->spd->data_bit;
    } else {
        ch_info->vc = ov16a10->cur_mode->vc[ch_info->index];
        ch_info->width = ov16a10->cur_mode->width;
        ch_info->height = ov16a10->cur_mode->height;
        ch_info->bus_fmt = ov16a10->cur_mode->bus_fmt;
    }
    return 0;
}

static long ov16a10_ioctl(struct v4l2_subdev *sd, unsigned int cmd, void *arg)
{
......
    case RKMODULE_GET_CHANNEL_INFO:
        ch_info = (struct rkmodule_channel_info *)arg;
        ret = ov16a10_get_channel_info(ov16a10, ch_info);
        break;
......
}
```

5. 注意点

此处有一个注意点容易引发一些异常，在有一些sensor全分辨率的输出的情况下，PD通道输出的可能会有一场，例如ov16a10，PD描述如下：

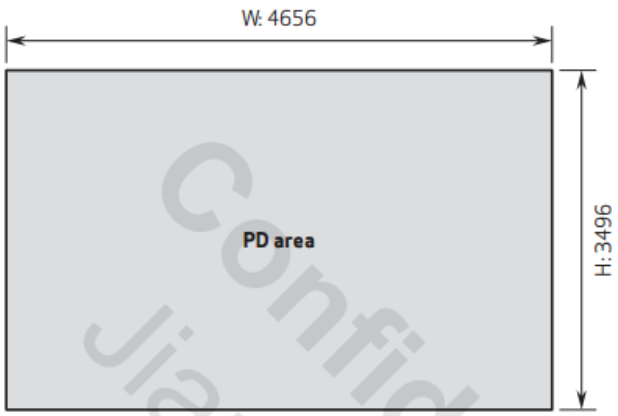


全分辨率按照如下描述：

3.6 PD pixel arrangement

PD pixels are evenly arranged in the PD area. The PD pixel amount is 508,668. PD area covers 100% of full size (4656 x 3496).

figure 3-4 PD pixel arrangement



根据上述的PD排列可以计算总的VC1的PD像素：

宽：4 / 16 x 4656，高：2 / 16 x 3496

根据datasheet了解：下述寄存器是配置一包发送多少个像素，也就是一行发送多少个像素：

3.5.2 PDAF data output: option 2 (MIPI virtual channel)

Option 2 is the MIPI virtual channel for PDAF raw data. Set registers 0x3664[0] to 1, 0x3664[2] to 1, and 0x3664[1] to 1 to enable PDAF output timing and set register 0x480E[2] to 1 to enable MIPI virtual channel mode. A normal image outputs through MIPI virtual channel 0. PDAF data outputs through MIPI virtual channel 1. PDAF data is output during image data horizontal blanking. Maximum PDAF data number in one package is configured by registers {0x4640, 0x4641}*8.

从上述寄存器看出必须是8的倍数，而4656/16 *4 = 1164不能被8整除，那么会导致最后一行PD像素不足，sensor没办法补足，会导致主控会收不下。

修改原始分辨率为：4608*3456，这样PD像素为1152x432，可以按照一包一包进行发送。

5.1.10 OTP添加

部分senor为了校准和优化最佳的图像效果，会烧录OTP的数据。因此sensor的驱动需要支持读取OTP数据，此章节简要介绍如何sensor的驱动中增加OTP的读取。由于OTP有不同的烧录规范，调试起来较为麻烦，因此我们要求统一按照RK提供的烧录规范烧录，驱动也基于RK的烧录规范实现。

RK的OTP驱动为otp_eeprom.c，实际使用过程中，只需要在DTS使能该设备。在sensor的驱动调用相应的接口获取OTP数据即可。

1. probe函数实现获取OTP设备

参考ov16880如下实现即可：

```
#ifdef RK_OTP
    eeprom_ctrl_node = of_parse_phandle(node, "eeprom-ctrl", 0);
    if (eeprom_ctrl_node) {
        eeprom_ctrl_client =
            of_find_i2c_device_by_node(eeprom_ctrl_node);
        of_node_put(eeprom_ctrl_node);
```

```
        if (IS_ERR_OR_NULL(eeprom_ctrl_client)) {
            dev_err(dev, "can not get node\n");
            goto continue_probe;
        }
        eeprom_ctrl = i2c_get_clientdata(eeprom_ctrl_client);
        if (IS_ERR_OR_NULL(eeprom_ctrl)) {
            dev_err(dev, "can not get eeprom i2c client\n");
        } else {
            otp_ptr = devm_kzalloc(dev, sizeof(*otp_ptr), GFP_KERNEL);
            if (!otp_ptr)
                return -ENOMEM;
            ret = v4l2_subdev_call(eeprom_ctrl,
                core, ioctl, 0, otp_ptr);
            if (!ret) {
                ov16880->otp = otp_ptr;
            } else {
                ov16880->otp = NULL;
                devm_kfree(dev, otp_ptr);
                dev_warn(dev, "can not get otp info, skip!\n");
            }
        }
    }
continue_probe:
#endif
```

2. 实现获取OTP的函数接口

此处按照RK的烧录规范，逻辑都是一致，直接参考ov16880的实现：

```
#ifdef RK_OTP
static void ov16880_get_otp(struct otp_info *otp,
                    struct rkmodule_inf *inf)
{
    u32 i, j;
    u32 w, h;

    /* awb */
    if (otp->awb_data.flag) {
        inf->awb.flag = 1;
        inf->awb.r_value = otp->awb_data.r_ratio;
        inf->awb.b_value = otp->awb_data.b_ratio;
        inf->awb.gr_value = otp->awb_data.g_ratio;
        inf->awb.gb_value = 0x0;

        inf->awb.golden_r_value = otp->awb_data.r_golden;
        inf->awb.golden_b_value = otp->awb_data.b_golden;
        inf->awb.golden_gr_value = otp->awb_data.g_golden;
        inf->awb.golden_gb_value = 0x0;
    }

    /* lsc */
    if (otp->lsc_data.flag) {
        inf->lsc.flag = 1;
        inf->lsc.width = otp->basic_data.size.width;
        inf->lsc.height = otp->basic_data.size.height;
        inf->lsc.table_size = otp->lsc_data.table_size;
```

```
        for (i = 0; i < 289; i++) {
            inf->lsc.lsc_r[i] = (otp->lsc_data.data[i * 2] << 8) |
                        otp->lsc_data.data[i * 2 + 1];
            inf->lsc.lsc_gr[i] = (otp->lsc_data.data[i * 2 + 578] << 8) |
                         otp->lsc_data.data[i * 2 + 579];
            inf->lsc.lsc_gb[i] = (otp->lsc_data.data[i * 2 + 1156] << 8) |
                         otp->lsc_data.data[i * 2 + 1157];
            inf->lsc.lsc_b[i] = (otp->lsc_data.data[i * 2 + 1734] << 8) |
                        otp->lsc_data.data[i * 2 + 1735];
        }
    }

    /* pdaf */
    if (otp->pdaf_data.flag) {
        inf->pdaf.flag = 1;
        inf->pdaf.gainmap_width = otp->pdaf_data.gainmap_width;
        inf->pdaf.gainmap_height = otp->pdaf_data.gainmap_height;
        inf->pdaf.dcc_mode = otp->pdaf_data.dcc_mode;
        inf->pdaf.dcc_dir = otp->pdaf_data.dcc_dir;
        inf->pdaf.dccmap_width = otp->pdaf_data.dccmap_width;
        inf->pdaf.dccmap_height = otp->pdaf_data.dccmap_height;
        w = otp->pdaf_data.gainmap_width;
        h = otp->pdaf_data.gainmap_height;
        for (i = 0; i < h; i++) {
            for (j = 0; j < w; j++) {
                inf->pdaf.gainmap[i * w + j] =
                    (otp->pdaf_data.gainmap[(i * w + j) * 2] << 8) |
                    otp->pdaf_data.gainmap[(i * w + j) * 2 + 1];
            }
        }
        w = otp->pdaf_data.dccmap_width;
        h = otp->pdaf_data.dccmap_height;
        for (i = 0; i < h; i++) {
            for (j = 0; j < w; j++) {
                inf->pdaf.dccmap[i * w + j] =
                    (otp->pdaf_data.dccmap[(i * w + j) * 2] << 8) |
                    otp->pdaf_data.dccmap[(i * w + j) * 2 + 1];
            }
        }
    }

    /* af */
    if (otp->af_data.flag) {
        inf->af.flag = 1;
        inf->af.dir_cnt = 1;
        inf->af.af_otp[0].vcm_start = otp->af_data.af_inf;
        inf->af.af_otp[0].vcm_end = otp->af_data.af_macro;
        inf->af.af_otp[0].vcm_dir = 0;
    }

}
#endif
```

3. 获取OTP数据

在上报模块信息给ISP的时候，添加OTP数据的上报：

```
static void ov16880_get_module_inf(struct ov16880 *ov16880,
                   struct rkmodule_inf *inf)
{
......
    if (otp)
        ov16880_get_otp(otp, inf);
......
}
```

5.1.11 虚拟通道配置

AHD转MIPI的应用场景中，通常需要多虚拟通道支持多路摄像头，需要在g_mbus_config确定通道：

```
static int nvp6188_g_mbus_config(struct v4l2_subdev *sd, unsigned int pad_id,
                 struct v4l2_mbus_config *cfg)
{
    cfg->type = V4L2_MBUS_CSI2_DPHY;
    cfg->flags = V4L2_MBUS_CSI2_4_LANE |
             V4L2_MBUS_CSI2_CHANNELS;

    return 0;
}
```

5.1.12 MIPI与并口配置

MIPI接口与DVP的接口在总线配置上有所不同，主要有如下不同：

g_mbus_config：需要指定是并口或者MIPI接口，若是MIPI接口需要指定type以及对应的lane数与通道数。

```
static int nvp6188_g_mbus_config(struct v4l2_subdev *sd, unsigned int pad_id,
                 struct v4l2_mbus_config *cfg)
{
    cfg->type = V4L2_MBUS_CSI2_DPHY;
    cfg->flags = V4L2_MBUS_CSI2_4_LANE |
             V4L2_MBUS_CSI2_CHANNELS;

    return 0;
}
```

若是DVP接口，需要指定type以及有效的边沿极性（必须正确指定）

```
static int nvp6158_g_mbus_config(struct v4l2_subdev *sd, unsigned int pad,
                 struct v4l2_mbus_config *cfg)
{
    struct nvp6158 *nvp6158 = to_nvp6158(sd);

    cfg->type = V4L2_MBUS_BT656;// V4L2_MBUS_PARALLEL
    if (nvp6158->dual_edge == 1) {
        cfg->flags = RKMODULE_CAMERA_BT656_CHANNELS |
            V4L2_MBUS_PCLK_SAMPLE_RISING |
            V4L2_MBUS_PCLK_SAMPLE_FALLING;
```

```
    } else {
        cfg->flags = RKMODULE_CAMERA_BT656_CHANNELS |
            V4L2_MBUS_PCLK_SAMPLE_RISING;
    }
    return 0;
}
```

若是BT656或者BT1120的接口，还需要在v4l2_subdev_video_ops实现.querystd成员，参考如下：

```
static int nvp6158_querystd(struct v4l2_subdev *sd, v4l2_std_id *std)
{
    struct nvp6158 *nvp6158 = to_nvp6158(sd);

    if ((nvp6158->mode > BT656I_TEST_MODES) &&
        (nvp6158->mode < NVP6158_DVP_MODES_END)) {
        /* for vicap detect bt1120 */
        *std = V4L2_STD_ATSC;
    } else {
        *std = V4L2_STD_PAL;
    }
    return 0;
}
```

若是BT656或者BT1120接口，还必须实现RKMODULE_GET_BT656_MBUS_INFO，BT656/BT1120都是调用这个ioctl，接口兼容，具体参考如下：

```
static __maybe_unused void
nvp6158_get_bt656_module_inf(struct nvp6158 *nvp6158,
                    struct rkmodule_bt656_mbus_info *inf)
{
    memset(inf, 0, sizeof(*inf));
    inf->flags = RKMODULE_CAMERA_BT656_PARSE_ID_LSB;
    switch (nvp6158->ch_nums) {
    case 1:
        inf->flags |= RKMODULE_CAMERA_BT656_CHANNEL_0;
        break;
    case 2:
        inf->flags |= RKMODULE_CAMERA_BT656_CHANNEL_0 |
                  RKMODULE_CAMERA_BT656_CHANNEL_1;
        break;
    case 4:
        inf->flags |= RKMODULE_CAMERA_BT656_CHANNELS;
        break;
    default:
        inf->flags |= RKMODULE_CAMERA_BT656_CHANNELS;
    }
}
```

5.1.13 VCM驱动移植

VCM即音圈马达，其主要原理是在一个永久磁场内，通过改变马达内线圈的直流电流大小，来控制弹簧片的拉伸位置，从而带动上下运动。手机摄像头广泛的使用VCM实现自动对焦功能，通过VCM可以调节镜头的位置，呈现清晰的图像。此章节简要介绍VCM马达驱动的移植。以DW9763为例进行介绍。

1. 从马达规格书获取控制时序图和控制模式

datesheet一般都有描述马达所支持的控制模式和控制时序，如DW9763就支持，Direct mode和SAC mode，SAC mode分为SAC1，SAC2，SAC2.5，SAC3，SAC4，具体的模式选择有寄存器可以配置：

SAC[2:0] : SAC mode select

SAC[2:0]	Mode	SAC[2:0]	Mode
000	SAC1	100	N/A
001	SAC2	101	SAC4
010	SAC2.5	110	N/A
011	SAC3	111	N/A

控制时序一般如下：

① 设置Ringing control寄存器：这个寄存器设置是direct mode还是ringingmode

② 设置具体的SAC mode和DIV

③ 设置Tvib

④ 向寄存器设置DAC

2. probe函数实现

① RK私有资源定义，命名方式如rockchip,camera-module-xxx，主要是提供设备参数和Camera设备进行匹配。VCM参数定义，命名方式如rockchip,vcm-xxx，主要涉及硬件参数启动电流、额定电流、移动模式，参数跟马达移动的范围和速度相关。VCM关键参数如下表所示

名称	描述
启动电流	VCM 刚好能够推动模组镜头从模组镜头可移动行程最近端(模组远焦)移动，此时VCM driver ic的输出电流值定义为启动电流
额定电流	VCM刚好推动模组镜头至模组镜头可移动行程的最远端(模组近焦)，此时VCM driver ic的输出电流值定义为额定电流
VCM电流输出模式	VCM移动过程中会产生振荡,VCM driver ic电流输出变化需要考虑vcm的振荡周期，以便最大程度减小振荡，输出模式决定了输出电流改变至目标值的时间

② VCM v4l2设备以及media实体的初始化

entity.function的指定，与camera sensor驱动不同，马达驱动注册的需要指定为：MEDIA_ENT_F_LENS。

3. 实现v4l2子设备驱动

主要需要实现如下两个成员

```
struct v4l2_subdev_core_ops
struct v4l2_ctrl_ops
```

① v4l2_subdev_core_ops 成员实现

主要实现以下回调函数： .compat_ioctl32

成员	描述
RK_VIDIOC_VCM_TIMEINFO	camera_engine通过该命令获取此次镜头移动所需时间，据此来判断镜头何时停止以及CIS帧曝光时间段是否与镜头移动时间段有 重叠；镜头移动时间与镜头移动距离、VCM driver ic电流输出模 式相关。

② v4l2_ctrl_ops成员函数实现

主要实现以下回调函数

成员	描述
.g_volatile_ctrl	获取VCM驱动定义的控制ID
.s_ctrl	设置VCM驱动定义的控制ID

其中.s_ctrl回调函数实现以下ID：

成员	描述
V4L2_CID_FOCUS_ABSOLUTE	camera_engine通过该命令来设置和获取镜头的绝对位置， RK AF算法中将镜头整个可移动行程的位置参数定义为[0,64]。

参考dw9763，具体实现如下：

```
static int dw9763_set_pos(struct dw9763_device *dev_vcm,
    unsigned int dest_pos)
{
    int ret;
    unsigned int position = 0;

    if (dest_pos >= VCMDRV_MAX_LOG)
        position = dev_vcm->start_current;
    else
        position = dev_vcm->start_current +
               (dev_vcm->step * (VCMDRV_MAX_LOG - dest_pos));

    if (position > DW9763_MAX_REG)
        position = DW9763_MAX_REG;

    dev_vcm->current_lens_pos = position;
    dev_vcm->current_related_pos = dest_pos;

    ret = dw9763_set_dac(dev_vcm, position);
    v4l2_dbg(1, debug, &dev_vcm->sd, "%s: set position %d, dac %d\n",
         __func__, dest_pos, position);

    return ret;
}

static int dw9763_set_ctrl(struct v4l2_ctrl *ctrl)
{
    struct dw9763_device *dev_vcm = to_dw9763_vcm(ctrl);
    struct i2c_client *client = dev_vcm->client;
```

```
    unsigned int dest_pos = ctrl->val;
    long mv_us;
    int ret = 0;

    v4l2_dbg(1, debug, &dev_vcm->sd, "ctrl->id: 0x%x, ctrl->val: 0x%x\n",
        ctrl->id, ctrl->val);

    if (ctrl->id == V4L2_CID_FOCUS_ABSOLUTE) {

        if (dest_pos > VCMDRV_MAX_LOG) {
            dev_info(&client->dev,
                "%s dest_pos is error. %d > %d\n",
                __func__, dest_pos, VCMDRV_MAX_LOG);
            return -EINVAL;
        }

        ret = dw9763_set_pos(dev_vcm, dest_pos);

        dev_vcm->move_us = dev_vcm->vcm_movefull_t * 1000;

        v4l2_dbg(1, debug, &dev_vcm->sd,
            "dest_pos %d, move_us %ld\n",
            dest_pos, dev_vcm->move_us);

        dev_vcm->start_move_tv = ns_to_kernel_old_timeval(ktime_get_ns());
        mv_us = dev_vcm->start_move_tv.tv_usec +
                dev_vcm->move_us;
        if (mv_us >= 1000000) {
            dev_vcm->end_move_tv.tv_sec =
                dev_vcm->start_move_tv.tv_sec + 1;
            dev_vcm->end_move_tv.tv_usec = mv_us - 1000000;
        } else {
            dev_vcm->end_move_tv.tv_sec =
                    dev_vcm->start_move_tv.tv_sec;
            dev_vcm->end_move_tv.tv_usec = mv_us;
        }
    }

    return ret;
}
```

需要注意的是：RK AF算法将模组镜头整个可移动行程的位置参数定义为[0,64]，模组镜头整个可移动行程在VCM驱动电流上对应的变化范围为[启动电流，额定电流]，该函数中实现这2者间的映射换算关系。

5.1.14 参考文档

更多相关信息可以参考文档：Rockchip_Driver_Guide_VI_CN_v1.x.x.pdf
其中x.x均代表版本号；以最新为准；
文档位置：SDK/RKDocs/common/camera/HAL3 或https://redmine.rock-chips.com/documents/53 中将ISP2X_Doc_xxx.zip.xxxx下载解压

5.2 dts链路配置说明

5.2.1 设备配置

camera sensor、VCM、OTP等都是作为I2C子设备实现，在dts配置的时候都需要配置在对应的i2c总线下，以下具体介绍每个设备在i2c都是如何配置的。

5.2.1.1 MIPI sensor

MIPI camera设备的配置需要注意点：

- 挂载的i2c总线与i2c地址需要配置正确，i2c地址此处为7为地址不包括读写位，sensor的datasheet提供的一般是8位地址，此处需要转换为7位地址配置。
- clk的配置，若是主控提供的，需要根据原理图设计正确配置，并且正确引用pinctrl，否则可能会导致clk不输出。若是外部晶振，则无需配置。
- powdn、reset控制配置：需要注意极性的配置，此处极性需要与代码逻辑流程想对应。
- 供电配置：此处是将电源使用regulator的形式，在驱动代码中进行控制。
- rk私有资源配置如下所示

> rockchip,camera-module-index：指定camera sensor的设备ID号，后续安卓注册camera会以这个作为cameraID
>
> rockchip,camera-module-facing：指定camera sensor的前后摄
>
> rockchip,camera-module-name：指定摄像头模组名称，后续会与效果文件匹配
>
> rockchip,camera-module-lens-name：指定摄像头模组镜头名称，后续需与效果文件匹配

- lens-focus、flash-leds、eeprom-ctrl分别引用VCM设备、闪光灯设备以及OTP设备。
- data-lanes：mipi的lane数配置，与实际硬件设计匹配，驱动可通过此处获取属性。
- 电源域配置：以RK3568/RK3566的主控示例，需要配置电源域power-domains = <&power RK3568_PD_VI>;

```
&i2c5 {
    status = "okay";
    pinctrl-names = "default";
    pinctrl-0 = <&i2c5m3_xfer>;

    ov16880: ov16880@36 {
        status = "okay";
        compatible = "ovti,ov16880";
        reg = <0x36>;
        //mclk配置
        clocks = <&cru CLK_MIPI_CAMERAOUT_M2>;
        clock-names = "xvclk";
        pinctrl-names = "default";
        pinctrl-0 = <&cam_clk2m0_clk2>;
        //powdn、reset配置
        power-domains = <&power RK3576_PD_VI>;
        pwdn-gpios = <&gpio3 RK_PD5 GPIO_ACTIVE_HIGH>;
        reset-gpios = <&gpio3 RK_PA7 GPIO_ACTIVE_HIGH>;
        //供电配置
        avdd-supply = <&vcc_mipipwr>;
        dovdd-supply = <&vcc_1v8_cam>;
```

```
        //dvdd-supply = <&vcc1v2_dvp>;
        //私有配置，index指定cameraID
        rockchip,camera-module-index = <0>;
        rockchip,camera-module-facing = "back";
        //以下两个属性会被aiq获取，用于匹配效果文件参数
        rockchip,camera-module-name = "KYT-11379-V1";
        rockchip,camera-module-lens-name = "default";
        //引用vcm设备
        lens-focus = <&ces6301>;
        //引用闪光灯设备
        flash-leds = <&flash_rgb13h>;
        //引用otp设备
        eeprom-ctrl = <&otp_eeprom>;
        //链路配置与下一章节相对应
        port {
            ov16880_out0: endpoint {
                remote-endpoint = <&mipi_in_ov16880>;
                //lane数配置，驱动代码可从此处获取mipi lane
                data-lanes = <1 2 3 4>;
            };
        };
    };
};
```

5.2.1.2 DVP sensor

DVP的sensor配置与MIPI相似，重复部分此处不再赘述，这里仅列举区别不同点以及注意事项。

- pinctrl：需要引用相关的gpio做iomux复用，若缺少配置导致无法接收数据。

```
&i2c2 {
    status = "okay";
    pinctrl-names = "default";
    pinctrl-0 = <&i2c2m4_xfer>;

    nvp6158: nvp6158@30 {
        compatible = "nvp6158-v4l2";
        status = "okay";
        reg = <0x30>;
        clocks = <&cru CLK_CIFOUT_OUT>;
        clock-names = "xvclk";
        power-domains = <&power RK3588_PD_VI>;
        pinctrl-names = "default";
        pinctrl-0 = <&cif_clk &cif_dvp_clk &cif_dvp_bus8 &cif_dvp_bus16>;
        pwr2-gpios = <&gpio1 RK_PA5 GPIO_ACTIVE_HIGH>;
        rst-gpios = <&gpio1 RK_PA3 GPIO_ACTIVE_HIGH>;
        rockchip,camera-module-index = <0>;
        rockchip,camera-module-facing = "back";
        rockchip,camera-module-name = "default";
        rockchip,camera-module-lens-name = "default";
        port {
            nvp6158_out: endpoint {
                remote-endpoint = <&dvp_in_bcam1>;
            };
        };
    };
```

```
};
```

5.2.1.3 VCM设备

VCM一般与sensor挂载在同一个i2c总线下面，VCM配置的时候需要正确配置启动电流额定电流等参数，如果配置错误，可能会影响对焦效果。同时index属性必须与相应的sensor想匹配。

```
&i2c5 {
    status = "okay";
    pinctrl-names = "default";
    pinctrl-0 = <&i2c5m3_xfer>;

    ces6301: ces6301@c {
        compatible = "chipextra,ces6301";
        status = "okay";
        reg = <0x0c>;
        //供电配置
        avdd-supply = <&vcc_mipipwr>;
        //最大电流
        rockchip,vcm-max-current = <120>;
        //启动电流
        rockchip,vcm-start-current = <10>;
        //额定电流
        rockchip,vcm-rated-current = <85>;
        //运行模式
        rockchip,vcm-step-mode = <9>;
        //私有资源，与对应的sensor属性相匹配
        rockchip,camera-module-index = <0>;
        rockchip,camera-module-facing = "back";
    };
};
```

5.2.1.4 OTP设备

OTP只需要使能对应的设备即可，参考如下：

```
&i2c5 {
    status = "okay";
    pinctrl-names = "default";
    pinctrl-0 = <&i2c5m3_xfer>;

    otp_eeprom: otp_eeprom@50 {
        compatible = "rk,otp_eeprom";
        status = "okay";
        reg = <0x50>;
    };
};
```

5.2.1.5 HDR配置

如果需要调试默认用HDR模式，则需要在DTS的设备节点增加如下属性，指示sensor初始化HDR的模式为HDRx2，

驱动注册的时候会解析这个参数，如果不需要默认使用，一般不用配置。

```
rockchip,camera-hdr-mode = <5>;
```

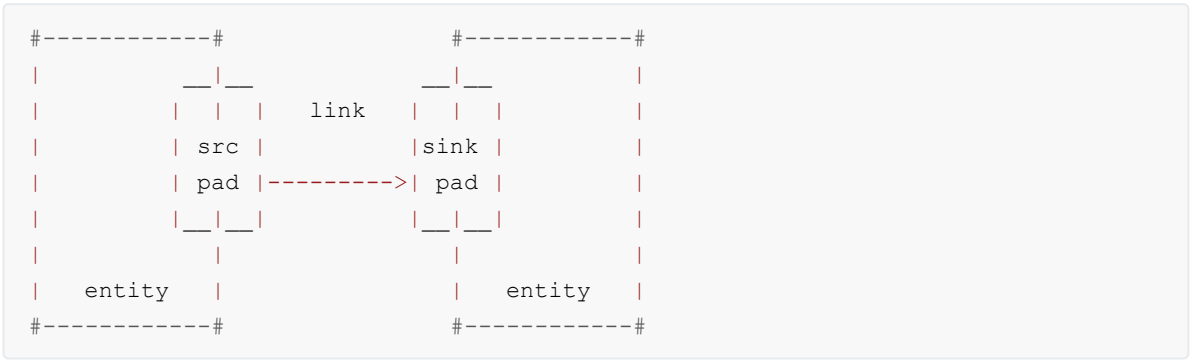
5.2.1.6 PDAF配置

PDAF需要在DTS的设备节点增加spd-id的配置，指示PD像素输出的VC通道，驱动注册的时候需要解析这个参数。

```
rockchip,spd-id = <1>;
```

5.2.2 链路配置

驱动框架数据流需要经过sensor->mipi-dphy->csi host->vicap->rkisp等模块处理，驱动上将各个模块虚拟成entity的示例，因此在DTS配置的时序我们需要正确数据链路的方向，示意图如下所示。



不同芯片平台，不同的sensor类型，其对应的链路配置也有所不同，以下具体介绍。

5.2.2.1 不同平台的数据链路

5.2.2.1.1 ISP平台

主要有RK3399、RK3288、RK3326等平台，在第二章节的数据流中，已经有不同版本的数据流框图，ISP1.0的版本较为简单，sensor输出MIPI数据流经过DPHY后直达ISP处理，在主控端最多经过两个硬件模块，如下所示：

```
//MIPI
sensor ---> dphy ---> rkisp
//DVP
sensor ---> rkisp
//DVP YUV sensor也可以通过cif模块接收
sensor ---> cif
```

在ISP2.x以及ISP3.x，cif硬件模块修改，数据链路也发生变化，各个芯片平台也有差异

5.2.2.1.2 ISP2.x平台

RK3568/RK3566 描述如下所示

```
//MIPI RAW有两种通路，直通模式直达isp，回读模式经过vicap
sensor ---> dphy --->  rkisp
sensor ---> dphy ---> csi host --->  vicap ---> ddr ---> rkisp
//MIPI YUV有两种通路，ISP与vicap均可以处理YUV数据
sensor ---> dphy --->  rkisp
sensor ---> dphy ---> csi host --->  vicap
//DVP RAW需要使用回读模式，经由vicap后isp处理
sensor --->  vicap ---> ddr ---> rkisp
//DVP YUV直接由vicap处理
sensor --->  vicap
```

5.2.2.1.3 ISP3.x

RK3588/RK3562/RK3576/RK3572与RK3568有所不同，按照如下描述所示

```
//MIPI RAW通路只有一种，都需经过vicap到isp，但有回读与直通两种模式
sensor ---> dphy ---> csi host --->  vicap ---> rkisp
sensor ---> dphy ---> csi host --->  vicap ---> ddr ---> rkisp
//MIPI YUV只能由vicap处理
sensor ---> dphy ---> csi host --->  vicap
//DVP RAW与MIPI类似，只是不需要经过dphy与csi
sensor --->  vicap ---> rkisp
sensor --->  vicap ---> ddr ---> rkisp
//DVP YUV直接由vicap处理
sensor --->  vicap
```

5.2.2.1.4 DPHY与DCPHY

RK3568/RK3566/RK3588/RK3562/RK3576/RK3572均有DPHY接口，如2.2接口规格所述，DPHY有4组data lane以及2组clk lane，因此作为4lane的一路使用，也可以拆分为2组2lane使用，以此拓展出更多的应用场景。但需要注意的是RK3568/RK3566只有一路CSI HOST，所以当DPHY拆分为两个2lane使用的时候，必须一路经csi到isp，一路直达isp，如下所示

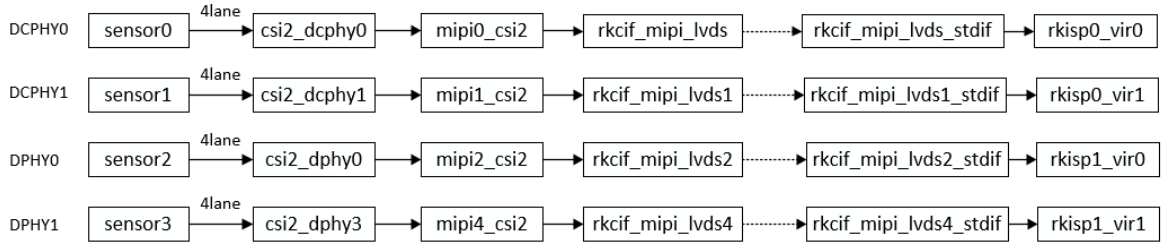
```
sensor ---> dphy1 ---> rkisp
sensor ---> dphy2 ---> csi host --->  vicap ---> rkisp
```

其余RK3588/RK3562/RK3576/RK3572则注意DPHY的拆分即可

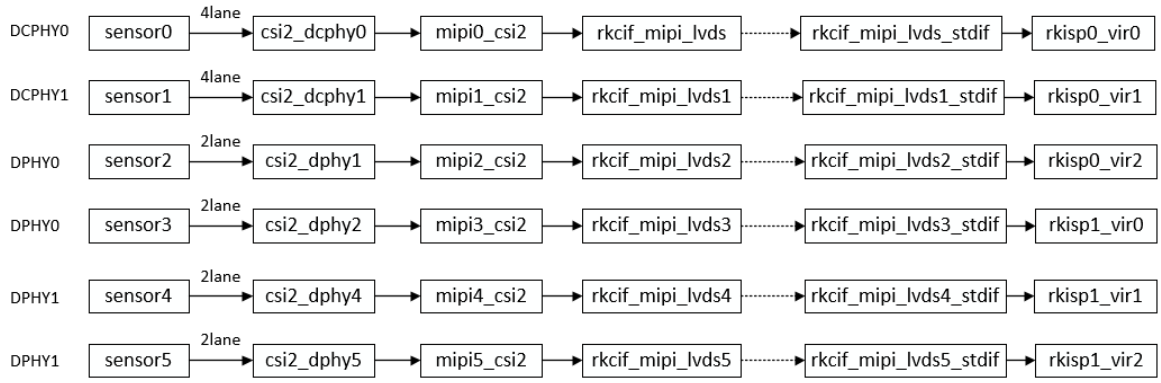
DCPHY只有一路clk lane无法进行拆分。

5.2.2.2 RK3588

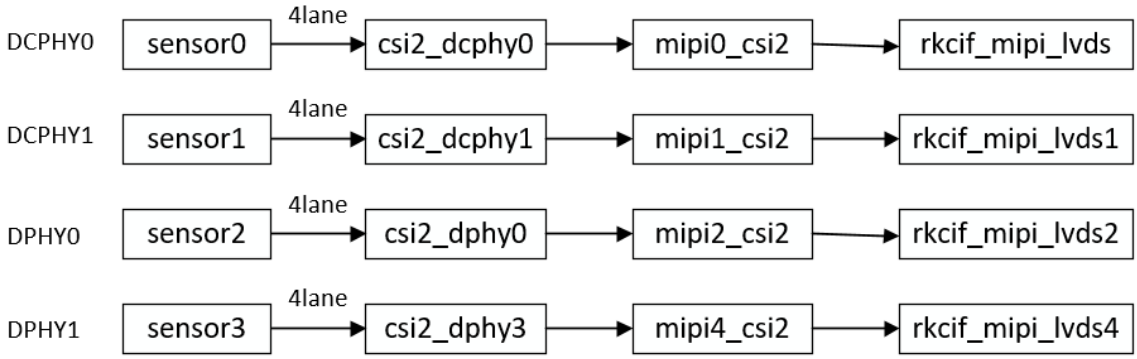
RK3588 4lane RAW的的链路示意图如下所示



将DPHY拆分为2lane+2lane的场景使用，示意图如下所示



RK3588 YUV 的场景只需要到rkcif一级即可，上述示意图虚线后续的不需要配置，参考如下：



RK3588 DVP的数据链路图如下所示，YUV的场景仅需要虚线箭头后级都不配置。



- DTS链路配置的时候csi2_dphy节点、mipi_csi2节点、rkcif_mipi_lvds、rkcif_mipi_lvds_stdif节点需要严格按照上述的示意图进行配置，rkisp_vir节点，可以按需取用，与前面的设备节点没有捆绑关系，如果多摄的场景，是前后摄切换的话，建议配置到同一个rkisp节点下方。若是按照上述示意图配置，则便按照多摄模式进行处理，需要在xml中打开多摄开关，后续进行介绍。
- RK3588支持两个dcphy，节点名称分别为csi2_dcphy0/csi2_dcphy1。每个dcphy硬件支持RX/TX同时使用，对于camera输入使用的是RX。支持DPHY/CPHY协议复用；需要注意的是同一个dcphy的TX/RX只能同时使用DPHY或同时使用CPHY。
- RK3588支持2个dphy硬件，这里我们称之为dphy0_hw/dphy1_hw，两个dphy硬件都可以工作在full mode 和split mode两种模式下。

    1. dphy0_hw 和 dphy1_hw

        1. full mode：节点名称使用csi2_dphy0 和 csi2_dphy3，最多支持4 lane。

2. split mode： 拆分成2个phy使用，分别为csi2_dphy1（使用0/1 lane）、csi2_dphy2(使用2/3 lane)，dphy1_hw 则拆分成csi2_dphy4和csi2_dphy5，每个phy最多支持2 lane。
3. 当dphy0_hw使用full mode时，链路需要按照csi2_dphy1这条链路来配置，但是节点名称csi2_dphy1需要修改为csi2_dphy0，软件上是通过phy的序号来区分phy使用的模式。dphy1_hw同理。

- 使用上述mipi phy节点，需要把对应的物理节点配置上。（csi2_dcphy0_hw/csi2_dcphy1_hw/csi2_dphy0_hw/csi2_dphy1_hw）
- 每个mipi phy都需要一个csi2模块来解析mipi协议，节点名称分别mipi0_csi2~mipi5_csi2
- RK3588所有camera数据都需要通过vicap，再链接到isp。rk3588仅支持一个vicap硬件，这个vicap支持同时输入6路mipi phy，及一路dvp数据，所以我们将vicap分化成rkcif_mipi_lvds~rkcif_mipi_lvds5、rkcif_dvp等7个节点，各个节点的绑定关系需要严格按照框图的节点序号配置。
- 每个vicap节点与isp的链接关系，通过对应虚拟出的XXX_sditf来指明链接关系。
- RK3588支持2个isp硬件，每个isp设备可虚拟出多个虚拟节点，软件上通过回读的方式，依次从ddr读取每一路的图像数据进isp处理。对于多摄方案，建议将数据流平均分配到两个isp上。
- RK3588 rkisp unit模式：两个ISP共同处理模式，sensor的图像大小超过16M的场景使用该模式，其余场景不需要使用。

RK3588参考DTS为：

rk3588-evb1-imx415.dtsi

rk3588s-evb1-lp4x-v10-camera.dtsi

5.2.2.3 RK3588S2说明

RK3588S2相对RK3588S，减少了一路DCPHY，增加了引出一路DPHY，驱动代码上做了兼容，这里描述一下配置上的关键点：

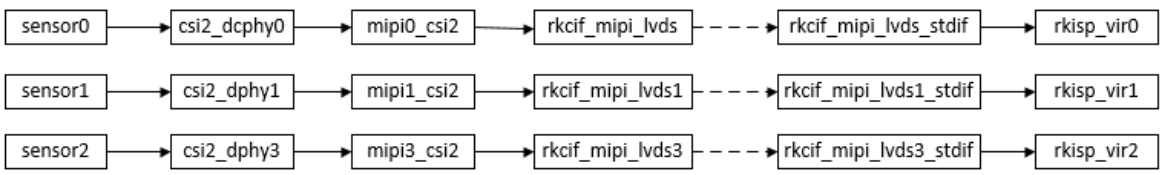
- 如果原先使用的项目是RK3588S，已经正常调通，只是将芯片替换为RK3588S2，则DTS配置上不需要修改，直接就可以使用，部分SDK过旧，驱动需要增加兼容补丁，详见redmine补丁简报。
- 如果是RK3588S2的原理图，原理图上接的是DPHY0，DTS则需要配置成DCPHY0（MIPI0）的

  如果原理图是RK3588S2的DPHY1，在DTS则需要配置成DPHY0（MIPI2）的配置

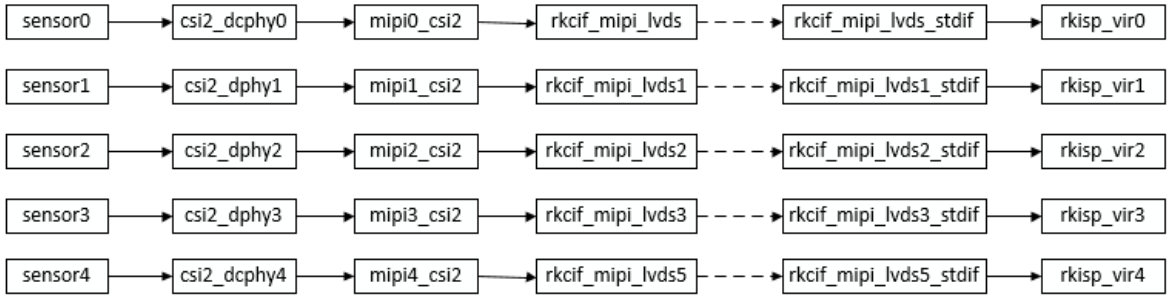
5.2.2.4 RK3576

RK3576链路的配置原理大体与RK3588类似，只是节点下标与硬件对应不同，此处提供RK3576的链路图，其余参考RK3588即可。

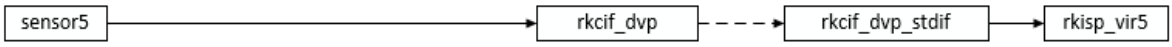
RK3576 4lane场景的数据链路图：



RK3576 拆分2lane场景的数据链路图：



RK3576 DVP数据链路图



RK3576DTS配置可参考SDK：

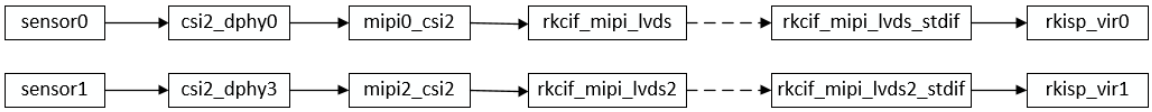
rk3576-rk806-tablet-camera.dtsi

rk3576-evb1-cam-4x.dtsi

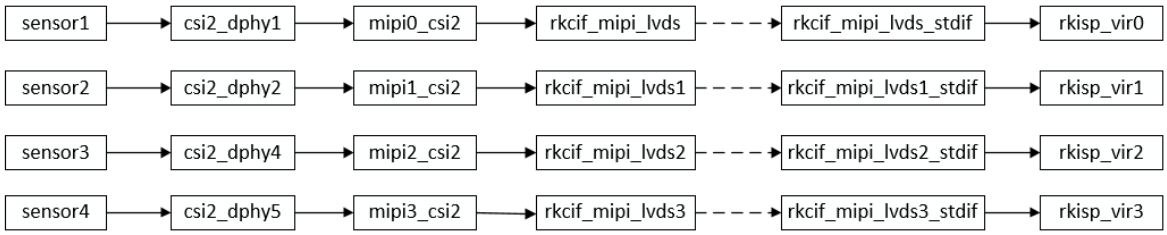
5.2.2.5 RK3572

RK3572链路的配置原理大体与RK3588类似，只是节点下标与硬件对应不同，此处提供RK3572的链路图，其余参考RK3588即可。

RK3572 4lane场景的数据链路图：



RK3572 拆分2lane场景的数据链路图：



RK3572 DVP 的链路配置同RK3576。

RK3572 DTS配置可参考SDK：

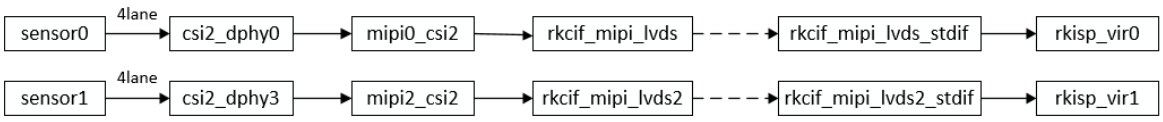
rk3572-evb1-cam-csi0.dtsi

rk3572-evb1-dual-cam-csi0.dtsi

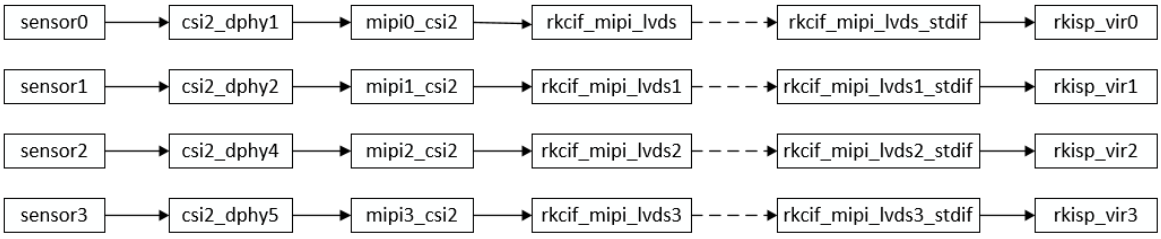
5.2.2.6 RK3562

RK3576链路的配置原理大体与RK3588类似，只是节点下标与硬件对应不同，需注意RK3562不支持DVP接口，其余可参考RK3588的描述。

RK3562 4lane场景数据链路图：



RK3562 2lane场景数据链路图：



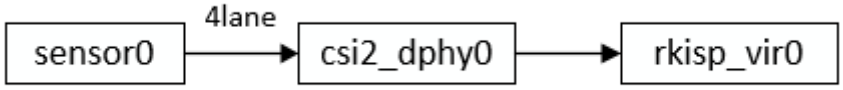
DTS配置可以参考SDK：

rk3562-rk817-tablet-camera.dtsi

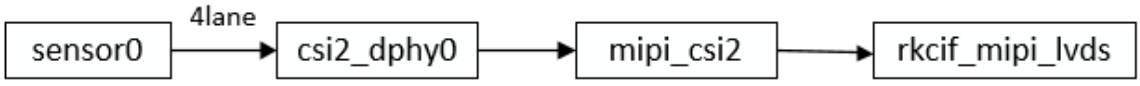
rk3562-evb1-cam.dtsi

5.2.2.7 RK3568/RK3566

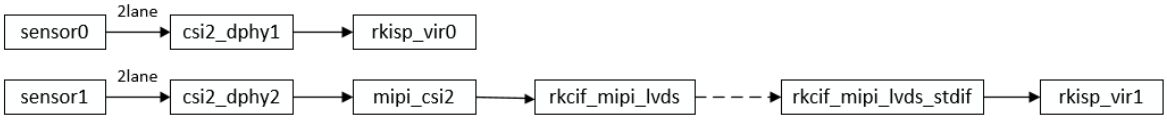
RK3568/RK3566只有一个4lane的DPHY，4lane时数据链路如下：



若sensor输出的是YUV的图像数据，例如HDMI-IN转MIPI-CSI、AHD转MIPI-CSI等场景，上述链路也可以配置为：



用作2lane+2lane的双摄场景，数据链路如下所示：



DVP的数据链路如下图：



DTS配置可以参考：

常用平板配置：rk3566-rk817-tablet.dts

双摄场景：rk3568-evb1-dual-camera.dtsi

HDMI-IN转MIPI-CSI场景：rk3568-evb2-lp4x-v10.dtsi

5.2.2.8 RK3399/RK3288/RK3326

ISP1.0版本直接参考前述描述的数据链路即可，配置参考：

rk3399-evb-ind-lpddr4-android-avb.dts

5.2.2.9 回读模式与直通模式

在RK3566/68之后的平台中，因为数据链路中vicap与ISP均有将数据存储到DDR的功能，因此引申出回读与直通的概念，以下对此解释。

- 直通：指数据经过vicap采集，直接发送给isp处理，不存储到ddr。需要注意的是hdr直通时，只有短帧是真正的直通，长帧需要存在ddr，isp再从ddr读取。
- 回读：指数据经过vicap采集到ddr，应用获取到数据后，将buffer地址推送给isp，isp再从ddr获取图像数据。
- dts配置时，一个isp硬件，如果只配置一个虚拟节点，默认使用直通模式，如果配置了多个虚拟节点默认使用回读模式。
- 直通模式，RK的平台要求sensor输出的vblank必须大于1ms，建议实际中大于1.5ms。

5.3 驱动功能验证

5.3.1 media拓扑确认

可以通过以下命令查看media链路拓扑图

```
media-ctl -p -d /dev/mediaX

X为media设备编号，建议将所有的链路都打出。
```

打印出的数据链路拓扑需要符合5.2.2章节中描述的数据链路，才算是链路成功注册。

5.3.2 v4l2输出数据流

开启图像数据流命令如下所示，需要根据实际情况，设置video节点、分辨率、格式等信息，video节点可以按照如下方法确认：使用上述查看media拓扑命令找到转接芯片所在的media节点并查看对应的pipeline，pipeline中节点名为：stream_cif_mipi_id0所对应的video节点既是取流的节点：

```
v4l2-ctl --verbose -d /dev/video0 --set-fmt-
video=width=1920,height=1080,pixelformat='NV12' --stream-mmap=4
```

-d：指定设备名称

--set-fmt-video：设置分辨率，需和sensor输出分辨率一致，sensor当前分辨率可通过media-ctl -p -d /dev/mediaX查看。

pixelformat：输出数据格式

--stream-mmap：mmap buffer数量。

上述命令再正常场景下会返回打印帧率信息，若没有，则是异常，正常应该如下：

```
rk3588_s:/ # v4l2-ctl --verbose -d /dev/video0 --set-fmt-
video=width=1920,height=1080,pixelformat='NV12' --stream-mmap=4
VIDIOC_QUERYCAP: ok
VIDIOC_G_FMT: ok
VIDIOC_S_FMT: ok
Format Video Capture Multiplanar:
        Width/Height      : 1920/1080
        Pixel Format      : 'NV12'
        Field             : None
        Number of planes  : 1
        Flags             :
        Colorspace        : Default
        Transfer Function : Default
        YCbCr Encoding    : Default
        Quantization      : Default
        Plane 0           :
           Bytes per Line : 1920
           Size Image     : 3110400
VIDIOC_REQBUFS: ok
VIDIOC_QUERYBUF: ok
VIDIOC_QUERYBUF: ok
VIDIOC_QBUF: ok
VIDIOC_QUERYBUF: ok
VIDIOC_QBUF: ok
VIDIOC_QUERYBUF: ok
VIDIOC_QBUF: ok
VIDIOC_QUERYBUF: ok
VIDIOC_QBUF: ok
VIDIOC_STREAMON: ok
idx: 0 seq:      0 bytesused: 3110400 ts: 249.466825
idx: 1 seq:      1 bytesused: 3110400 ts: 249.483367 delta: 16.542 ms
idx: 2 seq:      2 bytesused: 3110400 ts: 249.500033 delta: 16.666 ms
idx: 3 seq:      3 bytesused: 3110400 ts: 249.516700 delta: 16.667 ms
idx: 0 seq:      4 bytesused: 3110400 ts: 249.533367 delta: 16.667 ms fps: 60.11
idx: 1 seq:      5 bytesused: 3110400 ts: 249.550034 delta: 16.667 ms fps: 60.09
idx: 2 seq:      6 bytesused: 3110400 ts: 249.566700 delta: 16.666 ms fps: 60.08
idx: 3 seq:      7 bytesused: 3110400 ts: 249.583367 delta: 16.667 ms fps: 60.06
idx: 0 seq:      8 bytesused: 3110400 ts: 249.600034 delta: 16.667 ms fps: 60.06
```

5.3.3 v4l2抓图验证

RK3588/RK3576/RK3562等可以从vicap节点处抓取raw数据：

```
v4l2-ctl -d /dev/video0 --set-fmt-video=width=1920,height=1080,pixelformat=RG10 -
-stream-mmap=4 --stream-count=1 --stream-to=/tmp/cap.raw --stream-skip=2
```

-d：指定设备名称

--set-fmt-video：设置分辨率，需和sensor输出分辨率一致，sensor当前分辨率可通过media-ctl -p -d /dev/mediaX查看。

pixelformat：输出数据格式

--stream-mmap：mmap buffer数量。

--stream-count：抓取的帧数，多帧也是存在同一文件。

--stream-to：指定存储路径。

--stream-skip：跳掉的帧数。

上述指令正常情况下，会将图像数据存储在 --stream-to指定的路径，可以通过adb将图像拉取出来验证。

5.4 camera3_profiles.xml注册文件配置

camera3_profiles.xml文件对应SDK目录下具体芯片平台的文件：

```
hardware/rockchip/camera/etc/camera/camera3_profiles_rk3xxx.xml
```

在设备上的路径为：

```
/vendor/etc/camera/camera3_profiles.xml
```

若是临时调试，可以采用adb替换文件的形式，但需要注意文件路径与文件名的正确性。

xml增加配置的时候可以参考其他已有的sensor，后续仅介绍关键的几个修改点。

5.4.1 设备注册名称与ID

xml中name参数与moduleid参数取决定着能否成功注册安卓camera设备，若该两项属性配置异常，则会导致

- **name**：需要与驱动名称一致，有大小写区别；
- **moduleId**：需要与驱动dts中配置的index一致；

5.4.2 分辨率配置

scaler.availableStreamConfigurations/scaler.availableMinFrameDurations/ scaler.availableStallDurations：需要正确配置预预览支持的分辨率以及帧率，此处的分辨率不能大于驱动实际输出的分辨率大小，如下所示：

```
            <scaler.availableStreamConfigurations value="
                BLOB,1920x1080,OUTPUT,
                BLOB,176x144,OUTPUT,
                YCbCr_420_888,1920x1080,OUTPUT,
                YCbCr_420_888,176x144,OUTPUT,
                IMPLEMENTATION_DEFINED,1920x1080,OUTPUT,
                IMPLEMENTATION_DEFINED,176x144,OUTPUT"/>
            <scaler.availableMinFrameDurations value="
                BLOB,1920x1080,33333333,
```

```
                BLOB,176x144,33333333,
                YCbCr_420_888,1920x1080,33333333,
                YCbCr_420_888,176x144,33333333,
                IMPLEMENTATION_DEFINED,1920x1080,33333333,
                IMPLEMENTATION_DEFINED,176x144,33333333" />
            <scaler.availableStallDurations value="
                                                BLOB,1920x1080,33333333,
```

同时在 <Hal_tuning_RKISP1>中需要指定sensor输出的分辨率大小：

```
<supportTuningSize value="3264x2448"/>
```

5.4.3 前后摄配置

xml中需要指定前后摄，如下属性指定：

```
<lens.facing value="FRONT"/>
```

5.4.4 旋转配置

xml可以设置旋转角度，如下所示：

```
<sensor.orientation value="0"/>
```

5.4.5 SOC模式与RAW模式

在<Sensor_info_RKISP1>中需要配置sensorType，一般SOC为YUVsensor，即不启动3A，RAW为RAW sensor，启动3A，在调试过程中，点亮的时候，还没新增效果文件的话，可以先使用SOC模式，RAW sensor在SOC模式下，显示会偏绿偏暗，可以通过V4L2命令控制曝光提高亮度。

```
<sensorType value="SENSOR_TYPE_RAW"/> <!-- SENSOR_TYPE_SOC or SENSOR_TYPE_RAW -->
```

5.4.6 HDR设置

若是sensor支持HDR模式，需要在xml中相应增加支持，配置如下：

```
<aiq.workingMode value="HDR2"/> <!-- NORMAL or HDR2 or HDR3 -->
```

5.4.7 多摄配置

若是多摄的场景，需要xml打开如下配置：

```
<aiq.multicamera value="true"/> <!-- true or false -->
```

5.5 初始效果文件生成

在完成xml配置正确之后，一般就可以在SOC模式下出图，可以进一步生成一份初始的json效果文件，用于初步点亮，我们需要按照如下步骤配置初步的效果文件，点亮摄像头，后续还需效果优化，则需要业务端向RK申请效果调试资源。

5.5.1 生成初始效果文件

可以参考SDK已有的效果文件，生成初始版本的效果文件，SDK中效果文件存在目录：

external/camera_engine_rkaiq/iqfiles，其中isp3x指RK3588，isp21指RK3568/RK3566，isp32_lite指RK3562，isp39指RK3576，调试的时候可以根据自己芯片平台与sensor型号，在对应的目录中，复制sensor型号最为接近的json文件作为初版文件，并修改文件名，文件名命名规则如下：

sensor驱动名字 + 下划线 + module name + 下划线 + lens name + .json

例如：ov16880_KYT-11379-V1_default.json

其中module name以及lens name根据dts的配置匹配：

```
    ov16880: ov16880@36 {
        status = "okay";
        compatible = "ovti,ov16880";
......
        rockchip,camera-module-index = <0>;
        rockchip,camera-module-facing = "back";
        rockchip,camera-module-name = "KYT-11379-V1";
        rockchip,camera-module-lens-name = "default";
......
```

也可以参考Rockchip_IQ_Tools_Guide_ISP中对应的文档进行调试。

5.5.2 分辨率、LSC修改

除了文件名修改之后，还需要正确分辨率以及LSC参数才可以用于点亮出图。

分辨率需要修改sensor_calib中的resolution参数，如下所示：

```
    "sensor_calib": {
        "resolution":   {
            "width":    4672,
            "height":   3504
        },
```

其他包含分辨率的信息的也需要对应修改，我们可以进行全局搜索替换(width)x(height)替换为(new width)x(new height)

LSC修改，主要修改name参数为当前的sensor分辨率，lsc_sect_size_x为width/16，lsc_sect_size_y为height/16

```
                        "lsc_v2":   {
                            "common":   {
                                "enable":   1,
                                "resolutionAll":    [{
                                        "name": "4224x3136",
                                        "lsc_sect_size_x":  [264, 264, 264, 264,
264, 264, 264, 264, 264, 264, 264, 264, 264, 264, 264, 264],
                                        "lsc_sect_size_y":  [196, 196, 196, 196,
196, 196, 196, 196, 196, 196, 196, 196, 196, 196, 196, 196]
                                    }],
                                "resolutionAll_len":    1
                            },
```

5.5.3 sensor info参数

关于sensor info参数填写的注意事项，详细可以参考文档：《Rockchip_Driver_Guide_VI_CN_v1.1.5.pdf》以及Rockchip_Tuning_Guide_ISP中对应的文档。

此模块参数需要与sensor 手册、驱动代码核对清楚，正确填写，才可进行下一步的效果调试，具体参数在Rockchip_Tuning_Guide_ISP的4.1.2.5章节已经详细描述，此处不再赘述。

5.6 GKI版本说明

5.6.1 驱动ko加载顺序

驱动采用ko模块加载的方式，需要注意模块的加载顺序要求，才能正常注册数据链路，否则可能会导致链路异常，而引发功能异常。

GKI的版本，默认SDK会提供加载顺序的参考，在文件vendor_ramdisk_modules.load，如有新增camera sensor驱动的，可以参考上述文件，放置到camera驱动的位置即可：

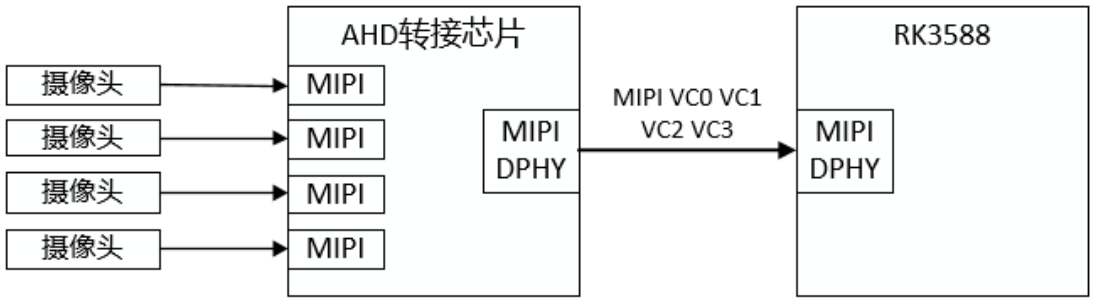
最新的kernel-6.1版本，camera通路部分的加载顺序参考如下所示：

```
phy-rockchip-samsung-dcphy.ko
phy-rockchip-csi2-dphy-hw.ko
videobuf2-cma-sg.ko
video_rkcif.ko
video_rkisp.ko
video_rkisp1.ko
rk_ircut.ko
leds-rgb13h.ko
dw9714.ko
ov13855.ko
ov13850.ko
vm149c.ko
dw9763.ko
gc8034.ko
imx415.ko
ov50c40.ko
gc05a2.ko
ov16880.ko
rk628-csi.ko
```

```
rk628-bt1120.ko
video-rk628.ko
phy-rockchip-csi2-dphy.ko
phy-rockchip-mipi-rx.ko
```

5.7 AHD/SERDES多路摄像头调试说明

车载的场景会使用AHD或者SERDES来实现多路摄像头同时接入的应用，通常是多路摄像头通过AD转接芯片接入，AD转接芯片将多路摄像头数据进行编码，通过MIPI 虚拟通道的方式整合成一路MIPI数据传输给RK3588等主控平台，由主控对数据进行解码拆分成4路数据。



RK的主控平台中，RK356X之后的平台才支持多路虚拟通道的方式，RK3399/RK3288/RK3326等主控平台不支持多路虚拟通道。在这个场景中，多数是YUV422的数据，故此处仅对YUV422的格式数据做介绍，在RK的camera驱动框架默认就支持该种方式，通过VICAP将数据收集，驱动会虚拟出对应的四个video节点，对应的节点entity分别为stream_cif_ mipi_id0~stream_cif_mipi_id3，对应vc0~vc3的数据，可通过V4L2从4个节点取得4路数据流。

5.7.1 驱动移植注意点

驱动可参考5.1驱动移植章节，需要注意多路配置需要打开虚拟通道，参考5.1.11章节。目前平台已支持或者验证的过的AHD转接芯片有NVP系列，如NVP6188、NVP6324（对应驱动jaguar1_drv），TP系列，如TP9950、TP2855等，RN系列的RN6752等等。

5.7.2 热拔插机制

5.7.2.1 AHD驱动实现热拔插检测

AHD转接芯片相比于常规的camera增加了热拔插的机制，以应对摄像头拔插的场景。该热拔插机制是未涉及分辨率的变化，因此热拔插机制在驱动实现。利用了VICAP的复位机制来实现。AHD的驱动需要实现热拔插的检测，并上报给VICAP驱动，由VICAP来实现数据流的复位。主要实现如下ioctl的成员：

成员名称	描述
RKMODULE_GET_VICAP_RST_INFO	VICAP从外设驱动获取复位信息
RKMODULE_SET_VICAP_RST_INFO	VICAP设置外设驱动复位信息
RKMODULE_GET_VC_HOTPLUG_INFO	获取热拔插信息，部分版本有用到，需要AHD驱动上报热拔插的状态。
RKMODULE_SET_QUICK_STREAM	快速开关数据流，此处需要实现，热拔插后由VICAP触发AHD驱动开关数据流。

用户可参考NVP6188的驱动实现，由热拔插检测线程持续检测前级摄像头的状态，并通过上述IOCTL成员上报VICAP驱动实现复位。

5.7.2.2 VICAP复位机制

VICAP复位机制除了可以在热拔插的场景使用，还可以检测断流、MIPI报错等场景进行异常复位。

5.7.2.2.1 kernel-4.19

kernel-4.19的版本中，VICAP复位机制通过DTS配置开启，需要在dts上面cif相关的接口设备节点，添加rockchip,cif-monitor参数，若dts不设置该参数，则默认不使能复位机制。

```
&rkcif_mipi_lvds {
    status = "okay";
    /* parameters for do cif reset detecting:
     * index0: monitor mode,
           0 for idle,
           1 for continue,
           2 for trigger,
           3 for hotplug (for nextchip)
     * index1: the frame id to start timer,
           min is 2
     * index2: frame num of monitoring cycle
     * index3: err time for keep monitoring
           after finding out err (ms)
     * index4: csi2 err reference val for resetting
     */
    rockchip,cif-monitor = <3 2 1 1000 5>;

    port {
        /* MIPI CSI-2 endpoint */
        cif_mipi_in: endpoint {
            remote-endpoint = <&mipi_csi2_output>;
            data-lanes = <1 2 3 4>;
        };
    };
};
```

对monitor的参数解释如下：

参数	描述
index0	用于说明复位的模式，目前主要有四种模式。
index1	对continue或者hotplug而言，在采集到index1帧数据后，触发监测定时器；
index2	监测定时器的周期，以一帧为单位，监测周期为index2帧；
index3	延时复位的时间参数，在发现vicap csi2 报错后，在该定义时间内，持续对监测，当检测到错误不再增加，进行复位，超过该定义时间，不管是否还在增加错误，都立即进行
index4	用于设定mipi csi err的出现次数，在达到该次数后，触发复位；

对index0的几种模式说明：

模式	描述
Idle	不启动复位模式
Continue	用于实时连续监测vicap是否mipi出错及断流，当发生出错及断流时进行vicap复位；监测定时器是在index1设定的帧数到达时，在帧尾进行初始化，进而开始监测，若未到达相应帧数，则不能触发监测；定时器以index2设定的周期进行采样检测；
Trigger	只在vicap出现csi2 协议层面的mipi报错时触发复位。通过index4来设定mipi 报错的次数，当达到index4的次数时，进而在帧尾中断触发监测定时器的初始化，在index2参数设定的周期到达后，实现一次vicap的复位；
Hotplug	热插拔模式，主要是针对类似n4/tp6188这类车机转接芯片实现的，用于解决插拔时图像割裂或者断流的问题；该模式具有continue模式的功能，即实时连续监测vicap是否mipi出错及断流，当发生出错及断流时进行vicap复位；监测定时器是在index1设定的帧数到达时，在帧尾进行初始化，进而开始监测，若未到达相应帧数，则不能触发监测；定时器以index2设定的周期进行采样检测；与continue的差别在于，在mipi 不报错和不断流的情况下，若是vicap所采集的sensor通过RKMODULE_SET_VICAP_RST_INFO命令置位复位使能，那么vicap在通过RKMODULE_GET_VICAP_RST_INFO获取到该信息后会触发复位操作。

较新的代码为了支持mipi/dvp都能够进行异常检测，对异常检测机制进行了修改，需要更新到包含以下提交：

```
commit 42245688955a6614ec5c5c73fe85fbc5b44fcc75
Author: Zefa Chen <zefa.chen@rock-chips.com>
Date:   Thu Jun 30 17:11:46 2022 +0800

    media: rockchip: rkcif reset all dev when trigger reset event

    move monitor param to rkcif node
    ag &rkcif {
            status = "okay";
            rockchip,cif-monitor = <3 200 1000 5 0>;
            /*
             * index 0: monitor mode
             *      val: 0 idle
             *           1 Continue mode, detect cut of stream or csi2 err
             *           2 trigger mode, detect csi2 err
             *           3 Hotplug, detect hot plug or cut off stream or csi2 err
             * index 1: monitor cycle
```

```
             *     val: timer of monitor cycle, unit ms
             * index 2: error interval
             *     val: time of error keeping, such as 1000,
             *          mean atfer one second error still occur,force reset dev,
             *          the max timer to wait error stop
             * index 3: err_ref_cnt:
             *     val: timer error ref val for reset
             * index 4: reset_by_user
             *     val: set val 1 to control reset by user
             */
    };

    Signed-off-by: Zefa Chen <zefa.chen@rock-chips.com>
    Change-Id: If4da872c7ee3c5140f94e5cf331cab84005361d2
```

其中：

1. rockchip，cif-monitor配置从rkcif_mipi_lvds挪到了rkcif节点
2. 删除原index1，采集n帧后才开启检测，改为启动数据流后就开始检测
3. 增加reset_by_user参数，置为1，表示驱动不主动做复位，等待应用检测后，自行决定是否做复位。应用定期轮训video节点的ioctl RKCIF_CMD_GET_RESET_INFO来获取复位状态，当状态为1表示需要复位，应用可在合适的时机下发ioctl RKCIF_CMD_SET_RESET，执行复位动作。

5.7.2.2.2 kernel-5.10/kernel-6.1

drivers/media/rockchip/cif/Kconfig增加异常检测的配置，可以直接修改默认值，或者在defconfig增加配置来覆盖掉默认值，具体参数如下：

参数	描述
ROCKCHIP_CIF_USE_MONITOR	是否启用异常监控机制，默认关闭，需要客户自行开启
ROCKCHIP_CIF_MONITOR_MODE	异常监测的模式，和kernel-4.19一致，参考前一小节
ROCKCHIP_CIF_MONITOR_START_FRAME	从哪一帧开始检测，建议配置成0，默认开启数据流就检测
ROCKCHIP_CIF_MONITOR_CYCLE	检测周期，以帧数为单位，当帧率为25fps，配置为4，则检测周期为40ms*4，根据项目自行评估检测间隔，建议400ms左右为一个检测周期
ROCKCHIP_CIF_MONITOR_KEEP_TIME	延时复位的时间参数，在发现vicap csi2 报错后，在该定义时间内，持续对监测，当检测到错误不再增加，进行复位，超过该定义时间，不管是否还在增加错误，都立即进行复位操作，时间单位ms；
ROCKCHIP_CIF_MONITOR_ERR_CNT	一般指mipi 报错统计次数，默认配置为5，表示统计到5*2次报错后触发复位
ROCKCHIP_CIF_RESET_BY_USER	当配置为1时，表示只检测异常，不主动复位，通知应用，由应用在合适的时机复位。

5.7.3 dts配置注意点

dts参考前述章节，驱动节点配置与pipeline的链路配置与YUV sensor无异，由VICAP接收数据。

5.7.3.1 热拔插配置

kernel-4.19的场景需要在dts配置打开使能，上一小节已描述。

5.7.3.2 适配cameraHAL

AHD 多路的场景只有一个驱动设备，一条pipeline链路，但是在HAL层需要模拟出4个camera设备给到应用层，因此需要适配到external cameraHAL，dts需要打开如下配置使能：

```
&rkcif {
    status = "okay";
    rockchip,android-usb-camerahal-enable;
};
```

5.7.4 external cameraHAL配置

上述配置打开之后，默认就会为一个AHD设备注册4路cameraID，但是在某些用户的场景，实际是两路，或者部分客户还有常规的MIPI camera接入就会导致HAL注册过多没用的cameraID，造成应用层开发的麻烦，HAL层支持屏蔽ID，可在device/rockchip/common/external_camera_config.xml文件中配置，文件的ignore参数可以设置跳过的video节点，例如用户驱动调试完整，可抓取数据流之后，可以设置非AHD的video屏蔽注册其cameraID，假设我的应用场景只有2路摄像头，AHD链路对应的video为video0~video3就会注册4个，我可以选择屏蔽video2、video3，在xml设置如下：

```
diff --git a/external_camera_config.xml b/external_camera_config.xml
index d377826..d982fb2 100755
--- a/external_camera_config.xml
+++ b/external_camera_config.xml
@@ -1,6 +1,8 @@
 <ExternalCamera>
     <Provider>
         <ignore> <!-- Internal video devices to be ignored by external camera
HAL -->
+        <id>2</id>
+        <id>3</id>
         <!-- <id>0</id> -->  <!-- No leading/trailing spaces -->
         </ignore>
     </Provider>
```

如果有常规的mipi camera，也是走camera3_profiles.xml进行注册camera，此处也可以添加其对应的video节点进行屏蔽。

6. 常用调试命令与调试工具

6.1 打开debug log开关

6.1.1 打开ISP驱动debug

```
echo 3 > /sys/module/video_rkisp/parameters/debug
echo 8 > /proc/sys/kernel/printk
```

6.1.2 打开VICAP驱动debug

```
echo 3 > /sys/module/video_rkcif/parameters/debug
echo 8 > /proc/sys/kernel/printk
```

6.1.3 打开cameraHAL debug

```
setprop persist.vendor.camera.hal.debug 5
```

6.1.4 打开rkaiq debug

AE的log

```
setprop persist.vendor.rkisp.log 0x1ff4
```

AWB的log:

```
setprop persist.vendor.rkisp.log 0x2ff4
```

AF的log:

```
setprop persist.vendor.rkisp.log 0x4ff5
```

AIQ打开所有模块的log：

```
setprop persist.vendor.rkisp.log 0x7fffffffff6
```

librkisp的log：

```
setprop persist.vendor.rkisp.log 0x1000000ff6
```

6.2 查看版本号

查看aiq的版本：

```
getprop | grep librkaiq.ver
示例结果：
1|rk3576_tablet:/ # getprop | grep librkaiq.ver
[vendor.cam.librkaiq.ver]: [AIQ v6.0x30.1]
```

6.3 常用V4L2命令

6.3.1 查看所有节点

```
grep -H '' /sys/class/video4linux/video*/name
grep -H '' /sys/class/video4linux/v4l-subdev*/name
```

6.3.2 抓图

```
v4l2-ctl -d /dev/video0 --set-fmt-video=width=1920,height=1080,pixelformat=RG10 -
-stream-mmap=4 --stream-count=1 --stream-to=/tmp/cap.raw --stream-skip=2
```

-d：指定设备名称

--set-fmt-video：设置分辨率，需和sensor输出分辨率一致，sensor当前分辨率可通过media-ctl -p -d /dev/mediaX查看。

pixelformat：输出数据格式

--stream-mmap：mmap buffer数量。

--stream-count：抓取的帧数，多帧也是存在同一文件。

--stream-to：指定存储路径。

--stream-skip：跳掉的帧数。

6.3.2.1 VICAP输出raw

1. 默认紧凑型，通过如下命令可以实现紧凑与非紧凑格式的切换：

```
echo 0 > /sys/devices/platform/rkcif_mipi_lvds/compact_test
```

其中，0表示非紧凑型，1表示紧凑型；对于同时使用多通道的设备或使用的通道不是vc0，命令可修改为

```
echo 0 0 0 0 > /sys/devices/platform/rkcif_mipi_lvds/compact_test
```

其中，echo后面的数字依次对应vc0、vc1、vc2、vc3通道的数据存储类型。

2. video0~3对应vc0~vc3。

3. 执行v4l2-ctl

```
v4l2-ctl -d /dev/video0 --set-fmt-video=width=1920,height=1080,pixelformat=RG10 -
-stream-mmap=4 --stream-count=1 --stream-to=/tmp/cap.raw --stream-skip=2
```

6.3.2.2 ISP maipath输出非紧凑raw

1. isp output format默认是YUYV8_2X8，使用如下命令切换到bayer raw格式：

```
media-ctl -d /dev/media0 --set-v4l2 '"rkisp-
ispsubdev":2[fmt:SBGGR12_1X12/2688x1520]'
```

注：rkisp-isp-subdev节点不一定在media0，-d指定设备，需要确认rkisp-isp-subdev在哪个 media节点

其中2表示pad2，source，详细说明自行查阅v4l2相关文档。 修改后必须media-ctl -p -d /dev/mediaX查看是否修改生效，生效后抓取的raw才是原始的raw数据。

2. 执行v4l2-ctl

```
v4l2-ctl -d /dev/video0 --set-fmt-video=width=1920,height=1080,pixelformat=RG10 -
-stream-mmap=4 --stream-count=1 --stream-to=/tmp/cap.raw --stream-skip=2
```

6.3.2.3 VICAP输出YUV

只支持输入端数据为YUV格式，若输入端为RAW格式，vicap无法输出YUV格式 。

```
v4l2-ctl -d /dev/video0 --set-fmt-video=width=1920,height=1080,pixelformat=NV12 -
-stream-mmap=4 --stream-count=1 --stream-to=/tmp/cap.raw --stream-skip=2
```

6.3.2.4 ISP输出YUV

```
v4l2-ctl -d /dev/video11 --set-fmt-video=width=1920,height=1080,pixelformat=NV12
--stream-mmap=4 --stream-count=1 --stream-to=/tmp/cap.raw --stream-skip=2
```

注意：

- 对于isp，可抓取mainpath或selfpath，video5只是举例，请按实际参数设置。
- ISP 输入数据是Raw情况下, ISP能够将Raw数据转换成YUV数据，同时包含了各项图像处理操作，此类图像处理操作需要RK AIQ来控制ISP的各项图像处理模块，当前命令只是数据流部分,图像处理模块参数采用驱动默认值，图像效果一般处于一个异常状态 。

6.3.3 输出数据流

```
v4l2-ctl --verbose -d /dev/video0 --set-fmt-
video=width=1920,height=1080,pixelformat='NV12' --stream-mmap=4
```

-d：指定设备名称

--set-fmt-video：设置分辨率，需和sensor输出分辨率一致，sensor当前分辨率可通过media-ctl -p -d /dev/mediaX查看。

pixelformat：输出数据格式

--stream-mmap：mmap buffer数量。

6.3.4 查看链路

可以通过以下命令查看media链路拓扑图

```
media-ctl -p -d /dev/mediaX

X为media设备编号，建议将所有的链路都打出。
```

6.3.5 查看sensor所有控制项

```
v4l2-ctl -d /dev/v4l-subdev2 --list-ctrls
```

/dev/v4l-subdev2为摄像头驱动对应的节点，可以通过6.3.1查找。

6.3.6 配置数据链路

所调试过程中需要修改数据链路的链接状态，可以通过对链路进行配置，参考如下：

```
media-ctl -d /dev/media1 -l '"m00_b_imx214 6-0010":0->"rockchip-csi2-dphy0":0[1]'
media-ctl -d /dev/media1 -l '"rockchip-csi2-dphy0":1->"rockchip-mipi-csi2":0[1]'
media-ctl -d /dev/media1 -l '"rockchip-mipi-csi2":1->"stream_cif_mipi_id0":0[1]'
media-ctl -d /dev/media1 -l '"rockchip-mipi-csi2":2->"stream_cif_mipi_id1":0[1]'
media-ctl -d /dev/media1 -l '"rockchip-mipi-csi2":3->"stream_cif_mipi_id2":0[1]'
media-ctl -d /dev/media1 -l '"rockchip-mipi-csi2":4->"stream_cif_mipi_id3":0[1]'
media-ctl -d /dev/media1 -l '"rockchip-mipi-csi2":5->"rkcif_scale_ch0":0[1]'
media-ctl -d /dev/media1 -l '"rockchip-mipi-csi2":6->"rkcif_scale_ch1":0[1]'
media-ctl -d /dev/media1 -l '"rockchip-mipi-csi2":7->"rkcif_scale_ch2":0[1]'
media-ctl -d /dev/media1 -l '"rockchip-mipi-csi2":8->"rkcif_scale_ch3":0[1]'
media-ctl -d /dev/media2 -l '"rkcif-mipi-lvds2":0->"rkisp-isp-subdev":0[1]'
media-ctl -d /dev/media2 -l '"rkisp-isp-subdev":2->"rkisp_mainpath":0[1]'
media-ctl -d /dev/media2 -l '"rkisp-isp-subdev":2->"rkisp_selfpath":0[1]'
media-ctl -d /dev/media2 -l '"rkisp-isp-subdev":3->"rkisp-statistics":0[1]'
media-ctl -d /dev/media2 -l '"rkisp-input-params":0->"rkisp-isp-subdev":1[1]'
```

6.3.7 配置ISP输出分辨率格式

```
media-ctl -d /dev/media0 --set-v4l2 '"rkisp-isp-subdev":0[fmt:UYVY2X8/3840x2160]'
media-ctl -d /dev/media0 --set-v4l2 '"rkisp-isp-subdev":0[crop:(0,0)/3840x2160]'
media-ctl -d /dev/media0 --set-v4l2 '"rkisp-isp-subdev":2[fmt:UYVY2X8/3840x2160]'
media-ctl -d /dev/media0 --set-v4l2 '"rkisp-isp-subdev":2[crop:(0,0)/3840x2160]'
```

6.3.8 手动设置曝光增益

可以通过手动设置曝光增益查看驱动是否正常

```
v4l2-ctl -d /dev/v4l-subdev2 --set-ctrl 'exposure=1216,analogue_gain=10'
```

也可以单独设置

```
v4l2-ctl -d /dev/v4l-subdev4 --set-ctrl exposure=1216
v4l2-ctl -d /dev/v4l-subdev4 --set-ctrl analogue_gain=10
```

exposure最大值被sensor vts限制住，最大限制条件可能是vts-4或vts-10，不同sensor根据sensor手册的说明做限制。假设当前帧率为30fps，最大曝光时间为33.3ms，要设置40ms的曝光，就得加大vts才能设置40ms的曝光，可以等比例换算，vts_30fps x 30fps = vts_25fps x 25fps，从而换算出25fps对应的vts，（vts - height）为vblank，将换算后vblank设置到sensor驱动即可设置更大的曝光，命令参考如下：

```
v4l2-ctl -d /dev/v4l-subdev4 --set-ctrl vertical_blanking=200
```

6.3.9 手动设置马达

在验证VCM驱动的时候，可以通过手动控制的方式，设置马达，来观察画面是否移动。

```
v4l2-ctl -d /dev/v4l-subdev3 --set-ctrl 'focus_absolute=64'
v4l2-ctl -d /dev/v4l-subdev3 --set-ctrl 'focus_absolute=0'
```

6.4 查看ISP/CIF状态命令

如下命令可查看isp与cif的状态

```
 cat /proc/rkcif*
 cat /proc/rkisp*
```

- rkcif状态信息说明

```
Driver Version:v00.02.00
Work Mode:ping pong
Monitor Mode:idle
aclk_cif:594000000
hclk_cif:198000000
dclk_cif:594000000
i0clk_cif:0
i1clk_cif:0
i2clk_cif:0
i3clk_cif:0
i4clk_cif:0
Input Info:
        src subdev:m00_b_rk628 5-0050
        interface:mipi csi2 dphy
        lanes:4
```

```
        hdr mode: normal
        format:RGB888_1X24/1920x1080@60
        crop.bounds:(0, 0)/1920x1080
Output Info:
        format:RGB3/1920x1080(0,0)
        compact:enable
        frame amount:36476392
        early:0 ms
        single readout:15 ms
        rate:16 ms
        fps:62
        irq statistics:
                        total:36476390
                        csi over flow:0
                        csi bandwidth lack:0
                        csi size err:2
                        not active buf cnt:0 0 0 0
                        all err count:0
                        frame dma end:36476390 0 0 0
irq time: 9778 ns
dma enable: 0x1 0x0 0x0 0x0
buf_cnt in drv: 6 0 0 0
total buf_cnt: 9 0 0 0

Group Info:
        Sync Mode:no sync
```

```
Work Mode：rv1109之后默认使用ping pong，建议使用ping pong模式。
Monitor Mode：监控模式，开启监控模式后，在mipi检查到异常的情况下，对vicap进行复位。
Input Info：输入端信息汇总
    src subdev:输入端设备，一般指sensor设备，包含摄像头朝向，索引号，设备名称，i2c总线，7bit slave地址等信息
    interface:数据物理接口，mipi、lvds、dvp等。
    vc channel：实际使用的vc通道，指mipi协议上的多通道传输的虚拟通道。
    hdr mode：sensor的工作模式，分为normal、 hdr_x2、hdr_x3。
    format：输入数据类型
    crop.bounds：裁剪参数，可在sensor驱动配置.get_selection，从而对输入源的数据适当裁剪。
Output Info：输出端信息汇总
    format:输出数据类型
    compact：默认紧凑型输出，相关定义查阅以下章节：如何抓取CIS输出的RAW、YUV数据
frame amount:
    early：wake up模式下，在采集到wait_line行数据后，提前将buffer送isp处理，默认模式是在采集到完整帧后送isp处理，这边显示的是相对采集到完整帧， buffer提前送isp的优化时间。
    single readout：hdr模式下，单帧传输时间，也就是长帧的传输时间。
    total readout：hdr模式下，长帧开始传输和短帧传输结束的时间差，也就是一帧合成帧的原始传输时间。
    rate：帧间隔时间。
    fps：帧率。
irq statistics:中断信息
    total：frame end + err 总的中断数
    csi over flow：overflow异常的中断数
    csi bandwidth lack：bandwidth lack 异常的中断数
    frame dma end：frame end的中断数，这个中断数等于从stream start开始，sensor输出的帧数。
```

- rkisp状态信息说明

```
130|rk3588s_s:/ # cat /proc/rkisp*
rkisp0-vir0 Version:v02.02.00
clk_isp_core 702000000
aclk_isp   750000000
hclk_isp   198000000
clk_isp_core_marvin 702000000
clk_isp_core_vicap 702000000
Interrupt  Cnt:1 ErrCnt:0
Input      rkcif-mipi-lvds Format:SBGGR10_1X10 Size:1920x1080@30fps Offset(0,0)
Isp online frame:0 working time:0ms v-blank:581363us
Output     rkisp_mainpath Format:NV12 Size:1920x1080 Dcrop(0,0|1920x1080)
(frame:0 rate:0ms delay:0ms frameloss:1)
Output     rkisp_selfpath Format:NV12 Size:1920x1080 Dcrop(0,0|1920x1080)
(frame:0 rate:0ms delay:0ms frameloss:2)
CMSK       OFF(0x0)
DPCC0      ON(0x40000005)
DPCC1      ON(0x40000005)
DPCC2      ON(0x40000005)
BLS        ON(0x1)
SDG        OFF(0x6197)
LSC        ON(0x5)
AWBGAIN    ON(0x6197) (gain: 0x01110111, 0x02b00150)
DEBAYER    ON(0xf000111)
CCM        ON(0x80000001)
GAMMA_OUT  ON(0x80000005)
CPROC      ON(0xf)
IE         OFF(0x0) (effect: BLACKWHITE)
HDRDRC     OFF(0x0)
HDRMGE     OFF(0x0)
BAYNR      ON(0xc0000001)
BAY3D      ON(0xa0012001)
YNR        ON(0xc4001001)
CNR        ON(0xa0000001)
SHARP      ON(0x40000001)
GIC        OFF(0x0)
DHAZ       ON(0x84110011)
3DLUT      OFF(0x0)
LDCH       OFF(0x0)
CSM        FULL(0x6197), y_offs:0x0 c_offs:0x80
           coeff Y:0x26 0x4b 0xf CB:0x1ea 0x1d6 0x40 CR:0x40 0x1ca 0x1f6
CAC        OFF(0x0)
GAIN       ON(0x80010111)
RAWAF      ON(0x107f)
RAWAWB     ON(0x4374fff7)
RAWAE0     ON(0x40000003)
RAWAE1     ON(0x400000f5)
RAWAE2     ON(0x400000f5)
RAWAE3     ON(0x40000005)
RAWHIST0   ON(0xc0000501)
RAWHIST1   ON(0xe0000501)
RAWHIST2   ON(0xe0000501)
RAWHIST3   ON(0xe0000501)
BigMode    OFF(0xe800018)
DEBUG1     space full status group (0x990fffff)
           ibuf2:0x9 ibuf1:0x9 ibuf0:0x0 mpfbc_infifo:0xf
           r1fifo:0xf r0fifo:0xf outfifo:0xf lafifo:0xf
DEBUG2     0x800c0000
```

```
            bay3d_fifo_full iir:1 cur:0
            module outform vertical counter:12, out frame counter:0
            isp output line counter:0
DEBUG3      isp pipeline group (0x95555554)
            mge(1 0) rawnr(0 1) bay3d(0 1) tmo(0 1)
            gic(0 1) dbr(0 1) debayer(0 1) dhaz(0 1)
            lut3d(0 1) ldch(0 1) ynr(0 1) shp(0 1)
            cgc(0 1) cac(0 1) isp_out(0 1) isp_in(0 0)
Monitor     OFF Cnt:0

Debug       mode:0x0
```

```
clk_isp:  isp 时钟频率
Interrupt：包含mipi中断、isp内各模块的中断，数据有递增，说明有数据进isp，ErrCnt错误中断统计信息
Input rkcif_mipi_lvds Format:SBGGR10_1X10 Size:2688x1520@30fps Offset(0,0)
输入源、输入格式、分辨率、帧率和裁剪信息
Isp Read mode:frame1 (frame:4061 rate:66ms idle time:10ms frameloss:6077)
cnt(total:2026 X1:1969 X2:56 X3:-1)
Read为回读模式（online对应直通模式）
frame1单帧线性（frame2 hdr2帧模式）
frame:帧号
rate:帧率，前后帧间隔
idle/working:isp硬件状态
time:isp硬件处理时间
frameloss:输入丢帧数
cnt: total:总回读次数 X1:1次回读次数 X2:2次回读次数 X3:3次回读次数
Output rkispp0 Format:FBC420 Size:2688x1520 (frame:4061 rate:66ms frameloss:2018)
Output rkisp_selfpath Format:FBCG Size:672x190 Dcrop(0,0|2688x1520) (frame:4061
rate:66ms delay:28ms frameloss:2017)
output输出流信息，输出buf有轮转才有更新信息
size:输出分辨率
Dcrop:输出裁剪信息
delay:对应时延信息（当前输出点时间搓-输入图像时间搓）
其他:  isp各个模块的开关状态
```

6.5 查看注册camera信息

可通过如下命令查看安卓注册的camera信息

```
dumpsys media.camera
```

6.6 cameraHAL抓图

可以通过如下方式dump camera在预览的时候，cameraHAL 的图像：

```
adb root
adb remount
adb shell setprop persist.vendor.camera.dump 139
adb shell setprop persist.vendor.camera.dump.cnt 20 设置dump20帧
adb shell setenforce 0
adb shell mkdir /data/dump
adb shell chmod 777 /data/dump/
adb shell setprop  persist.vendor.camera.dump.path /data/dump/
打开相机，导出/data/dump/
```

注意需要关闭：

```
adb shell setprop persist.vendor.camera.dump 0
```

6.7 查看camera预览帧率

查看预览帧率，可以通过logcat查看

```
setprop persist.vendor.camera.hal.debug 5
logcat | grep FPS
```

6.8 io命令使用

需要在rockchip_defconfig打开如下配置：

```
CONFIG_DEVMEM=y
```

另外还需要修改：

```
vim mkcombinedroot/configs/android-11.config
```

删除如下：

```
# CONFIG_DEVMEM is not set
```

如果需要完整编译安卓还需要修改其他依赖

6.9 raw/yuv图预览查看工具

YUV图可以通过7yuv工具查看：[7yuv Download Free (Windows) - 2.5 | Softpedia]

raw图可以使用RKISP_Tuner工具，或者使用ImageTool

6.10 rkaiq_demo抓图

调试过程中，V4l2工具只能抓取raw图，如果需要抓取3A之后的图像，可以使用rkaiq工具，SDK默认有编译该脚本，可以将该脚本push到设备：

```
android_13\out\target\product\rk3562_t\vendor\bin\rkaiq_demo
```

抓数据命令如下：

```
rkaiq_demo --device /dev/video11 --width 1920 --height 1080 --iqpath
/vendor/etc/camera/rkisp2/ --rkaiq --stream-skip=10 --stream-count=5 --stream-
to=/data/gc8034.yuv
```

--device：指定设备节点，一般指定rkisp的mainpath

--width：指定宽

--height：指定高

--iqpath：指定效果文件路径

--rkaiq：启动aiq

--stream-skip：设置跳帧

--stream-count：设置抓取的帧数

--stream-to：设置存储路径

6.11 3A server抓图

除了使用rkaiq_demo抓图，我们也可以启动3A server 服务，在使用V4L2的命令抓图，3A server启动命令如下：

```
rkaiq_3A_server
//多摄场景
rkaiq_multi_cam_3A_server
```

7. 常见问题排查方法

7.1 I2C通讯异常排查

驱动调试过程中，最常见的异常就是i2c通讯异常，通常表现为如下log：

```
[    1.973566][    T1] gc8034 4-0037: driver version: 00.01.09
[    1.973648][    T1] gc8034 4-0037: Failed to get power-gpios, maybe no use
[    1.973682][    T1] gc8034 4-0037: Failed to get reset-gpios
[    1.974133][    T1] gc8034 4-0037: lane_num(2)  pixel_rate(319887360)
[    1.979292][    T1] gc8034 4-0037: gc8034 read reg:0xf0 failed !
[    1.979461][    T1] gc8034 4-0037: gc8034 read reg:0xf1 failed !
[    1.979477][    T1] gc8034 4-0037: Unexpected sensor id(000000), ret(-6)
```

i2c通讯问题一般与sensor驱动、DTS配置相关，大多数是由于上电时序、gpio配置等异常造成，以下分析一下i2c通讯异常该从哪些方面进行排查：

1. i2c地址配置错误

①确认i2c地址是否配置正确

一般sensor的datasheet都会明确给出i2c的地址，需要注意的是，大多数sensor都有SID选择引脚，有两个i2c地址可以选择，这里需要硬件确认选择是哪个地址。如下ov16880描述，SID引脚可以选择i2c地址。

table 1-1 signal descriptions (sheet 1 of 3)

pad number	signal name	pad type	description
1	DVDD	power	digital circuit power
2	DOGND	ground	I/O ground
3	AGND	ground	analog ground
4	AVDD	power	analog power
5	SID	input	SCCB ID select
6	DOVDD	power	I/O power

2.10 serial camera control bus (SCCB) interface

The Serial Camera Control Bus (SCCB) interface controls the image sensor operation. Refer to the *OmniVision Technologies Serial Camera Control Bus (SCCB) Specification* for detailed usage of the serial control port.

In the OV16880, the SCCB ID is controlled by the SID pin, and can be programmable. If SID is low, the sensor's SCCB address comes from register 0x3035 which has a default value of 0x6C for write (0x6D for read). There is an alternative ID in register 0x3036, which has a default value of 0x42. If SID is high, the sensor's SCCB address comes from register 0x3037 which has a default value of 0x20 for write (0x21 for read).

②i2c总线地址是否正确

需要根据硬件原理图明确i2c是挂载在哪路i2c总线下，并在DTS正确配置。

③是否配置成7位地址

DTS中配置的i2c地址是7位地址，不包括最后的读写位，检查是否配置成8位地址，如上图展示的某sensor，其i2c地址有0x6c和0x20，这就是8位地址，在dts中需要配置为0x36或者0x10。

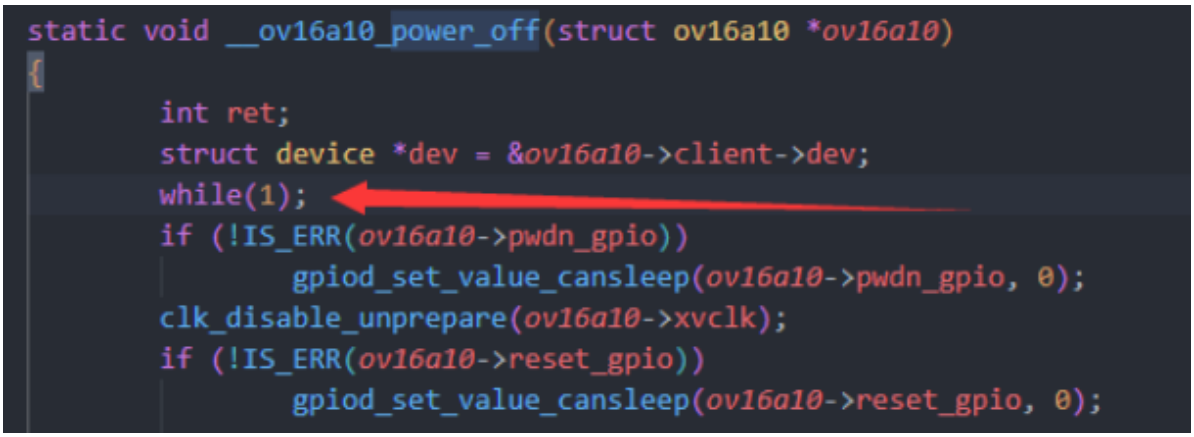
④i2c总线是否正确引用pinctrl

需要检查一下i2c的引脚是否正确的引用，可以先看原理图对应的gpio，在检查dts是否正确引用。如下示例引用i2c6m4_xfer。

```
&i2c6 {
    status = "okay";
    pinctrl-names = "default";
    pinctrl-0 = <&i2c6m4_xfer>;
```

2. 上电时序与MCLK时钟问题

上电时序与时钟的检查可以一起完成，sensor的驱动一般都是在使用的时候才会打开mclk和电源，因此出现问题的时候一半mclk和电源都是关闭的状态，我们可以让sensor不下电，或者直接卡死在下电的流程，然后再确认时钟和电源的问题：



```
static void __ov16a10_power_off(struct ov16a10 *ov16a10)
{
        int ret;
        struct device *dev = &ov16a10->client->dev;
        while(1);
        if (!IS_ERR(ov16a10->pwdn_gpio))
                gpiod_set_value_cansleep(ov16a10->pwdn_gpio, 0);
        clk_disable_unprepare(ov16a10->xvclk);
        if (!IS_ERR(ov16a10->reset_gpio))
                gpiod_set_value_cansleep(ov16a10->reset_gpio, 0);
```

①检查MCLK是否正常，电压幅度是否正确

一般sensor都是24M或者27M的clk，使用示波器确认clk的频率是否正确，如果测量不到24M的mclk，可以怀疑如下原因：

- sensor的驱动上电的时候是否正确打开clk，参考如下：

```
    ret = clk_set_rate(ov16a10->xvclk, OV16A10_XVCLK_FREQ);
    if (ret < 0)
        dev_warn(dev, "Failed to set xvclk rate (24MHz)\n");
    if (clk_get_rate(ov16a10->xvclk) != OV16A10_XVCLK_FREQ)
        dev_warn(dev, "xvclk mismatched, modes are based on 24MHz\n");
    ret = clk_prepare_enable(ov16a10->xvclk);
    if (ret < 0) {
        dev_err(dev, "Failed to enable xvclk\n");
        return ret;
    }
```

- gpio是否被其他模块占用了，可以使用io命令查询iomux复用是否正确，一般串口log也会打印相应的错误。
- MCLK的电压幅度不对，比较大的可能是电源域io-domain配置错误，有1.8V和3.3V，需要根据实际原理图进行配置io-domian。

②检查供电

sensor的供电一般是avdd，dvdd，diovdd三路电源，使用万用表确认3路电源是否正常。入不正常，可检查驱动代码是否有打开对应的电源，或者硬件设计错误。

③检查reset，powdn引脚状态

可以查看sensor的datasheet对这引脚的描述，确定是高电平有效还是低电平有效：示例如下的sensor说明reset PWDN都是低电平有效，那么说明低电平的时候会复位或者进入power down，因此需要正常工作的话，这两个引脚都应该是高电平的状态才正确。

22	DOGND	ground	I/O ground
23	XSHUTDOWN	input	reset and power down (active low with pull down resistor)
24	PWDNB	input	power down (active low)
25	TM	input	test mode (active high, input with pull down resistor)

如果引脚状态错误的话，一般是驱动代码和dts配置没有统一，修改其中一个配置即可。

容易出错的地方：dts配置的gpio的GPIO_ACTIVE_HIGH跟datasheet说的没关系，dts配置的高/低有效是给驱动代码使用的，如果是高有效，那么驱动代码写1则是输出高，写0输出低，反之同理。

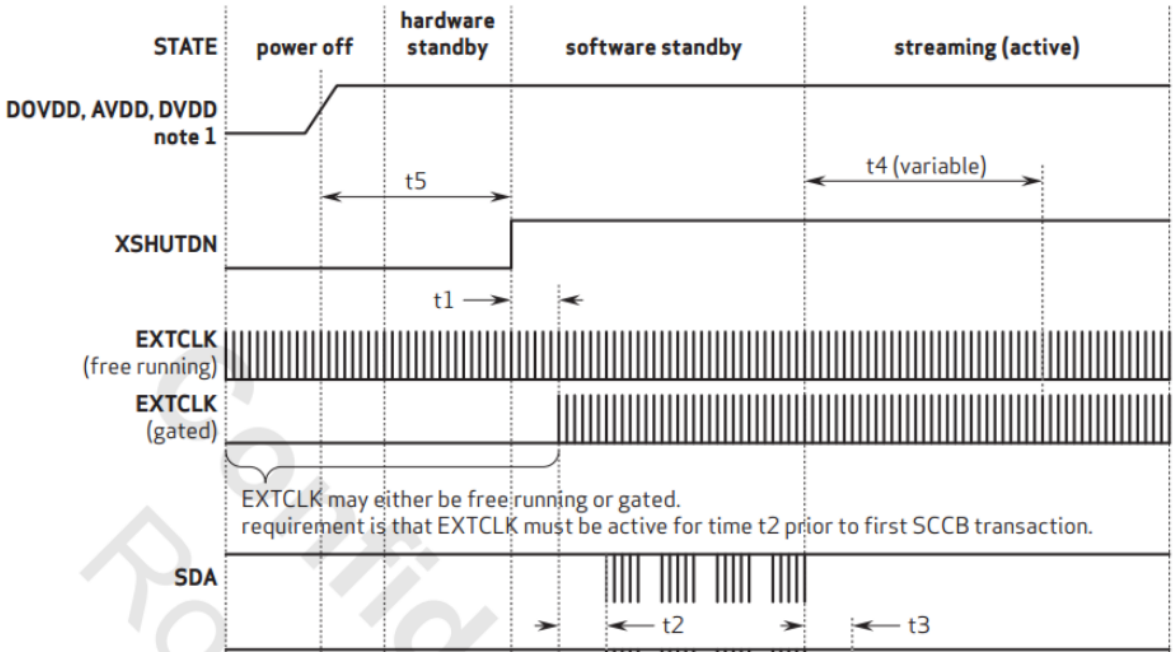
如果dts和驱动代码都操作正确，但是gpio始终无法改变，需要确认：

- gpio是否被其他的模块复用
- gpio是否默认上拉或者下拉，需要配置成pcfg_pull_none。

④检查上电时序

sensor的datasheet都会描述sensor需要的上电时序，驱动的控制需要符合上电时序要求，比较严格的sensor，这里更加容易出错：

figure 2-3 power up sequence



驱动上电函数如下：

```
static int __ov13855_power_on(struct ov13855 *ov13855)
{
    int ret;
    u32 delay_us;
    struct device *dev = &ov13855->client->dev;

    if (!IS_ERR(ov13855->power_gpio))
        gpiod_set_value_cansleep(ov13855->power_gpio, 1);

    usleep_range(1000, 2000);

    if (!IS_ERR_OR_NULL(ov13855->pins_default)) {
        ret = pinctrl_select_state(ov13855->pinctrl,
                    ov13855->pins_default);
        if (ret < 0)
```

```
            dev_err(dev, "could not set pins\n");
    }
    ret = clk_set_rate(ov13855->xvclk, OV13855_XVCLK_FREQ);
    if (ret < 0)
        dev_warn(dev, "Failed to set xvclk rate (24MHz)\n");
    if (clk_get_rate(ov13855->xvclk) != OV13855_XVCLK_FREQ)
        dev_warn(dev, "xvclk mismatched, modes are based on 24MHz\n");
    ret = clk_prepare_enable(ov13855->xvclk);
    if (ret < 0) {
        dev_err(dev, "Failed to enable xvclk\n");
        return ret;
    }
    if (!IS_ERR(ov13855->reset_gpio))
        gpiod_set_value_cansleep(ov13855->reset_gpio, 0);

    ret = regulator_bulk_enable(OV13855_NUM_SUPPLIES, ov13855->supplies);
    if (ret < 0) {
        dev_err(dev, "Failed to enable regulators\n");
        goto disable_clk;
    }

    if (!IS_ERR(ov13855->reset_gpio))
        gpiod_set_value_cansleep(ov13855->reset_gpio, 1);

    usleep_range(5000, 6000);
    if (!IS_ERR(ov13855->pwdn_gpio))
        gpiod_set_value_cansleep(ov13855->pwdn_gpio, 1);

    /* 8192 cycles prior to first SCCB transaction */
    delay_us = ov13855_cal_delay(8192);
    usleep_range(delay_us * 2, delay_us * 3);

    return 0;

disable_clk:
    clk_disable_unprepare(ov13855->xvclk);

    return ret;
}
```

⑤供电、CLK等正确，i2c依旧不通

有时候camera我们使用的是转接小板，需要使用主板对转接小板进行供电，这时候执行上电函数之后，主板给小板供电比较慢，可能会导致check ID失败，这时候可以在驱动的上电函数（power_on）适当的加上一些延时，保证小板成功供电，再执行check id的操作。

7.2 APK打不开预览排查

1. 确认驱动是否正常注册

查看log，类似如下，sensor check ID正常之后就会注册成V4L2子设备，log会打印连接到对应的DPHY上面。

```
[    2.441640][    T1] gc8034 4-0037: driver version: 00.01.09
[    2.441689][    T1] gc8034 4-0037: Failed to get power-gpios, maybe no use
[    2.441794][    T1] gc8034 4-0037: supply dovdd not found, using dummy
regulator
[    2.441902][    T1] gc8034 4-0037: supply dvdd not found, using dummy
regulator
[    2.441946][    T1] gc8034 4-0037: supply avdd not found, using dummy
regulator
[    2.441985][    T1] gc8034 4-0037: lane_num(2)  pixel_rate(320400000)
[    2.441998][    T1] gc8034 4-0037: could not get default pinstate
[    2.442006][    T1] gc8034 4-0037: could not get sleep pinstate
[    2.448076][    T1] gc8034 4-0037: detected gc8044 sensor
[    2.448156][    T1] rockchip-csi2-dphy csi2-dphy1: dphy1 matches m00_b_gc8034
4-0037:bus type 5
[    2.448806][    T1] rkcif-mipi-lvds: Async subdev notifier completed
[    2.449111][  T144] rkisp-vir0: Async subdev notifier completed
```

sensor注册失败，大多数原因都是check ID失败，也就是i2c不通，可以参考上一章节排查。

2. 查看拓扑是否正常

可使用如下命令查看拓扑链路，与5.2描述进行比对，查看是否符合预期：

```
media-ctl -p -d /dev/mediaX   X = 0..1..2..3...
```

3. 确认驱动是否可以抓图

可以使用V4L2命令抓取数据流，抓取raw图与YUV图，如果抓不到图，可以按照如下几种情况排查：

①命令敲完直接报错（no such devices/invalid param）

命令敲完直接停止报错，检查命令是否正确。可能还会出现no such devices之类的报错，大概率是由于链路没有

enabled的导致的，正常开机之后pipeline是enabled的，如果打开了apk，pipeline有可能会被清空，建议直接重启之

后不打开apk，直接抓数据。

②抓数据没有，串口也没有报错，抓图select timeout

这种情况可能性一般有以下：

- 接口错误，RK3588/RK3576等有多个MIPI接口，dts配置的接口和实际硬件的连接对不上
- 硬件错误，硬件的连接性错了
- sensor没输出
- 其他可能原因

可以先核对DTS的配置与硬件的连接是否符合预期。

③出现MIPI 相关报错

具体见7.3章节

④其他报错

包括PICK_SIZE_ERROR报错、iommu的报错等，需要确认一下sensor输出的图像格式是否正确，驱动配置的是否

正确，抓图的大小与图像格式是否正确等等。

4. 确认camera服务是否成功注册设备

如果驱动正常抓取数据流，则可以排除驱动的问题，则要进一步排查，可以通过如下命令查看是否成功注册camera:

```
dumpsys media.camera
```

如没有注册摄像头，则按照7.4的章节进行排查。

5. 检查SOC模式出图

如果能正常注册设备，可以将camera3_profiles.xml中修改为SOC模式进行测试，查看是否出图，SOC模式正常，在测试RW模式，RAW模式需要注意效果文件的有效性。

6. 确认效果文件的有效性

JSON文件在设备上的/vendor/etc/camera/rkisp2/，命名规则如下：

```
sensor_module-name_module-lens-name.json
```

确认文件命令是否正确，已经效果文件是否有效，需要跟dts的配置对应。

7. 其他HAL或者engine的报错

可以参考前述打开对应的debug开关，抓取logcat进行分析。

7.3 MIPI异常报错排查

出现MIPI相关的报错，类似如下CRC报错：

```
[ 2266.859853][    C0] mipi2-csi2 ERR1:0x1000000 (crc,vc: 0)
[ 2266.859862][    C0] mipi2-csi2 ERR1:0x1000000 (crc,vc: 0)
[ 2266.859869][    C0] mipi2-csi2 ERR2:0x100 (ecc,vc: 0)
[ 2266.859920][    C0] mipi2-csi2 ERR1:0x1000000 (crc,vc: 0)
[ 2266.859930][    C0] mipi2-csi2 ERR1:0x10000000 (ecc2)
[ 2266.859962][    C0] mipi2-csi2 ERR1:0x1000000 (crc,vc: 0)
[ 2266.859970][    C0] mipi2-csi2 ERR2:0x100 (ecc,vc: 0)
[ 2266.860026][    C0] mipi2-csi2 ERR1:0x1000000 (crc,vc: 0)
[ 2266.860033][    C0] mipi2-csi2 ERR1:0x10000000 (ecc2)
[ 2266.860093][    C0] mipi2-csi2 ERR1:0x1000000 (crc,vc: 0)
[ 2266.860101][    C0] mipi2-csi2 ERR1:0x1000000 (crc,vc: 0)
[ 2266.860108][    C0] mipi2-csi2 ERR2:0x100 (ecc,vc: 0)
[ 2266.860160][    C0] mipi2-csi2 ERR1:0x1000000 (crc,vc: 0)
[ 2266.860167][    C0] mipi2-csi2 ERR1:0x1000000 (crc,vc: 0)
```

出现上述mipi相关的报错，crc/ecc等报错，与phy层的mipi信号相关，可能原因如下：

- 驱动代码 link freq的配置，该配置为mipi clk，即速率的一半，需要与TX端的MIPI 速率对应配置

如果没有条件测试转接芯片的输出MIPI CLK，那么可以按照如下方式进行计算：

`MIPI CLK = Htotal * Vtotal * bpp * fps / lane num / 2`，例如1080P60根据公式计算如下：

`2200 * 1125 * 16 * 60 / 4 / 2 = 297M`。

- dts MIPI lane数是否配置错误，驱动接口g_mbus接口lane数是否配置错误，需要根据实际情况配置。

- MIPI信号质量问题，检查是否存在飞线，走线过长等问题。MIPI信号质量的问题，可以通过将驱动的xvclk降低到12M/6M，如果降低后没出现MIPI的报错，那么需要排查硬件的走线等等。MIPI时序的问题，建议测量MIPI信号，看是否符合spec。
- 如果接的是RK3588的DCPHY，检查MIPI的LP11时序是否符合DCPHY的设计要求。

7.4 注册不上摄像头排查

查看是否注册安卓摄像头命令如下：

```
dumpsys media.camera
```

如果没有成功注册摄像头，可以按照如下方式排查：

1. 检查camera3_profiles.xml配置

注册不上camera，大多数是由于xml没有配置成功导致的，认真核对name与moduleID的配置，具体参考5.4章节。

2. 头文件不匹配问题

camera注册的过程中，上层可能会通过一些ioctl获取sensor的信息，如果头文件对某些结构体的定义不一致，则会导致注册失败。类似开机注册的报错如下：

```
01-01 00:00:07.519   436   436 E RkCamera: <HAL> PlatformData: Fail to construct
iq default metadatas!
```

可以对比应用使用的头文件定义与内核的头文件定义差异之处，以内核为基准，修改相关的结构体即可，头文件所在目录分别如下：

```
kernel-5.10/include/uapi/linux/rk-camera-module.h
external/camera_engine_rkaiq/include/common/rk-camera-module.h
```

3. 驱动回调函数

报错信息与上述相同，还有可能是驱动缺少api回调造成的，与注册相关的驱动接口，常见于enum_frame_interval接口没有实现，实现对应的回调函数即可。

7.5 多摄配置打开异常

在多摄的场景，必须在对应的xml打开如下配置，否则会导致异常

```
<aiq.multicamera value="true"/> <!-- true or false -->
```

7.6 常见调试问题参考文档

其他更多相关的调试问题，可参考文档《Rockchip_Trouble_Shooting_CameraHAL3_CN》。